



Computer Systems and Professional Practice

Professor Matthew Leeke and Dr Ian Batten
School of Computer Science
University of Birmingham

Topic 4 - Memory & Storage Systems

Session Outline

The memory hierarchy, including motivation and structure

Organising memory cells for addressing

Detecting and correcting errors in unreliable memory and network communications

The design and implementation of common memory components, including hard disks and optical discs

The Memory Hierarchy

The Choice of Memory Technology

To construct a memory system, we must think carefully about the properties of each storage approach

Only condition is that a memory system must facilitate data modification (writing) and data detection (reading)

Many factors influence the choice of memory technology

- Frequency of access

- Access time

- Capacity required

- Cost, e.g., cost per bit

The Designer's Dilemma for Memory Systems

The designer's dilemma

Desire low cost and high capacity - leads to choice of cheap per bit, high capacity and slow access time

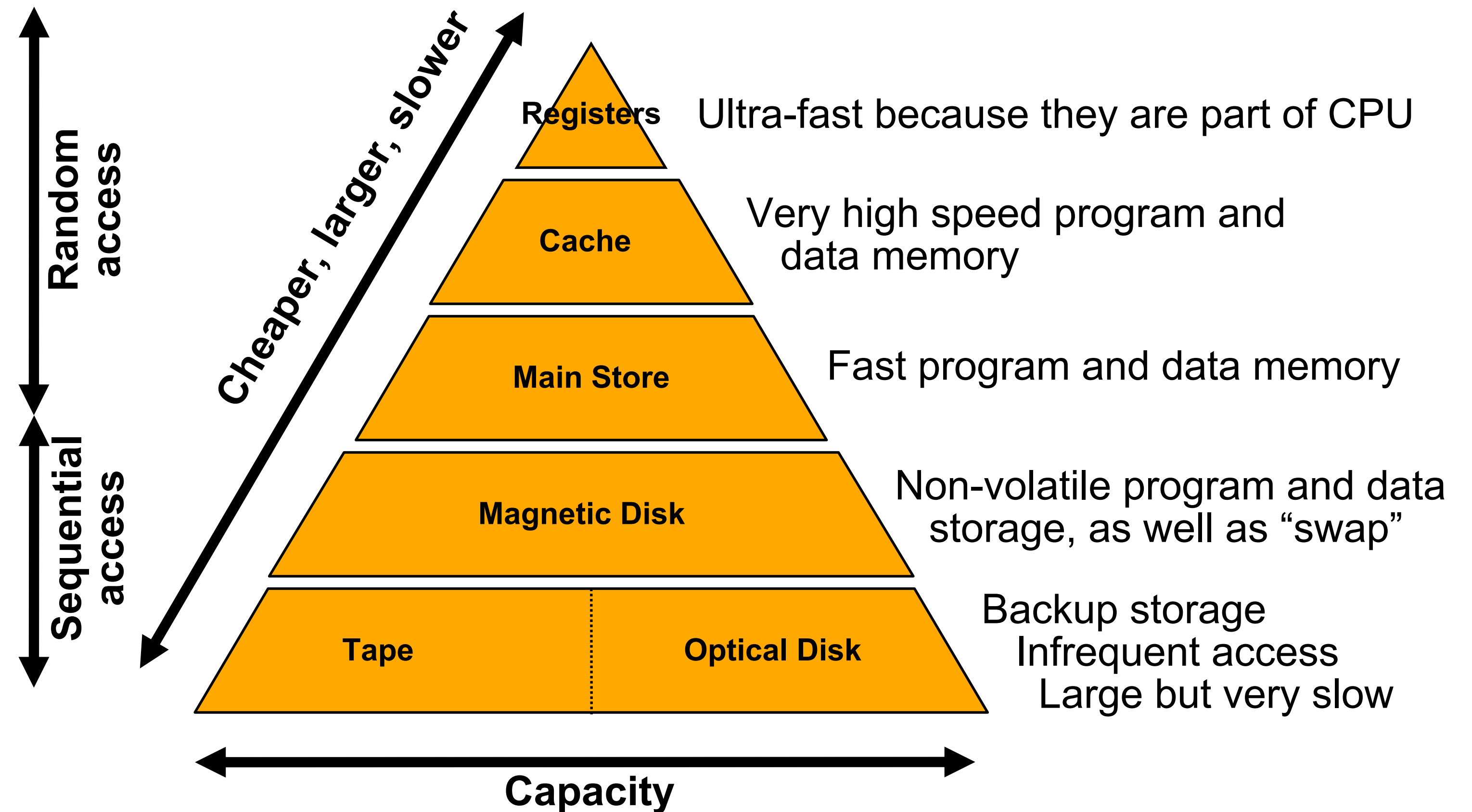
Desire high performance - leads to choice of expensive per bit, low capacity and fast access time

(The contra-cases, such as "what about technologies which are low capacity, slow and expensive?" are removed by market forces)

To solve the problem we choose both by organising memory into a hierarchy

The Memory Hierarchy

A general view of the memory system that underpin modern computer systems



Why Have a Memory Hierarchy?

Economics!

Small sequential loops, subroutines, operations on arrays, tables and other structures mean that the locations in memory which will be accessed is somewhat predictable

Temporal locality - If a particular memory location is referenced, it is likely that the same location will be referenced again in the near future

Spatial locality - If a particular memory location is referenced, it is likely that nearby memory locations will be referenced in the near future

Reasonable assumption - It has been shown that 90% of memory accesses are within ± 2 kilobytes of previous PC position

Semiconductor Memory Types

Particularly interested in memory that provides random access

RAM is most common type of semiconductor memory, though the term is misleading since all types shown above allow random access

Memory Type	Category	Erasure	Write Mechanism	Volatility
Random Access Memory (RAM)	Read-write	Electrically at byte-level	Electrically written	Volatile
Read-only Memory (ROM)	Read-only	Not possible	Mask written	Non-volatile
Programmable ROM (PROM)		Not possible	Electrically written	
Erasable PROM (EPROM)	Read-mostly	UV-light at chip-level		
Electrically Erasable PROM (EEPROM)		Electrically at byte-level		
Flash Memory		Electrically at block-level		

Cache Memory

What is a Cache?

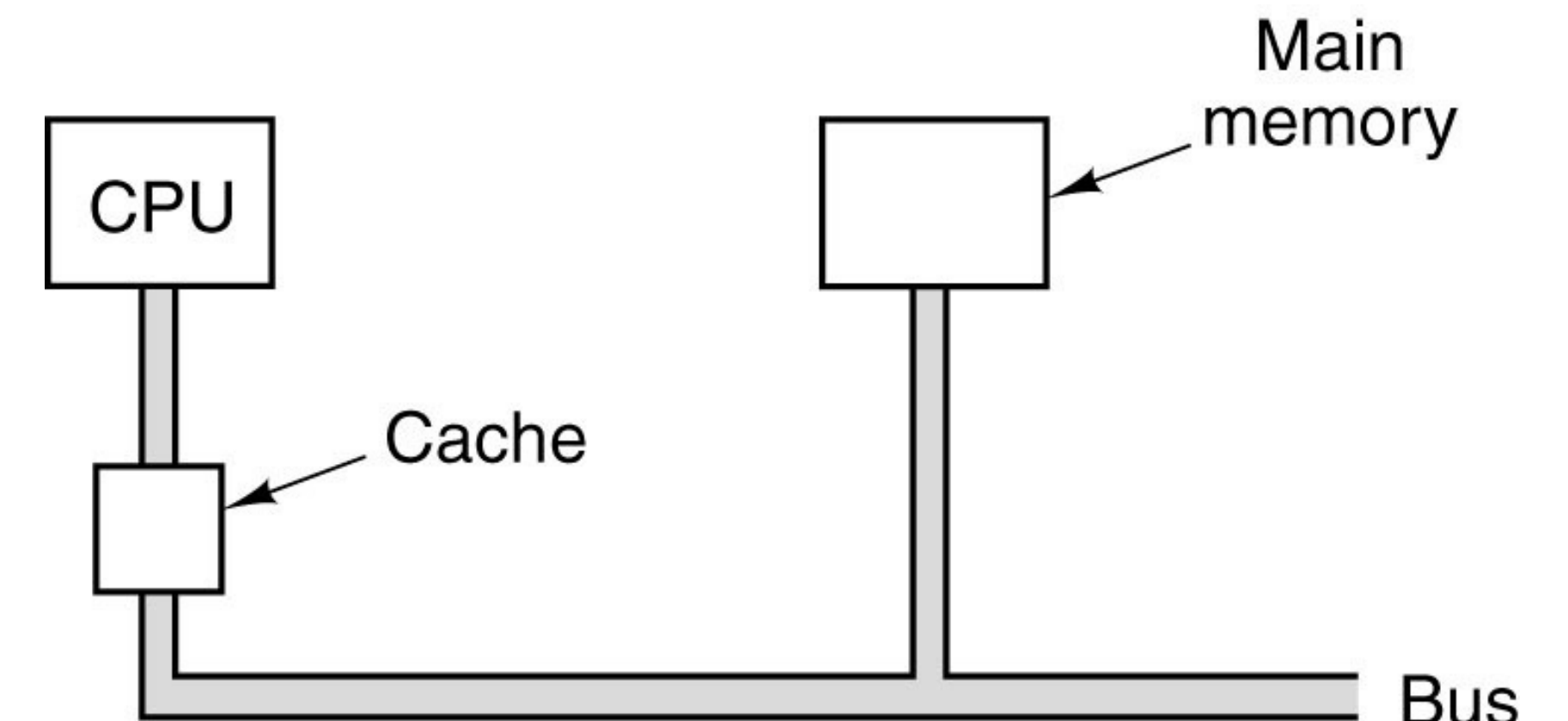
"90% of memory accesses within only 2 Kbytes"

So store those 2 Kbytes in a small, fast "cache" memory

If data required by CPU is in the cache (a "cache hit"), big speed improvement

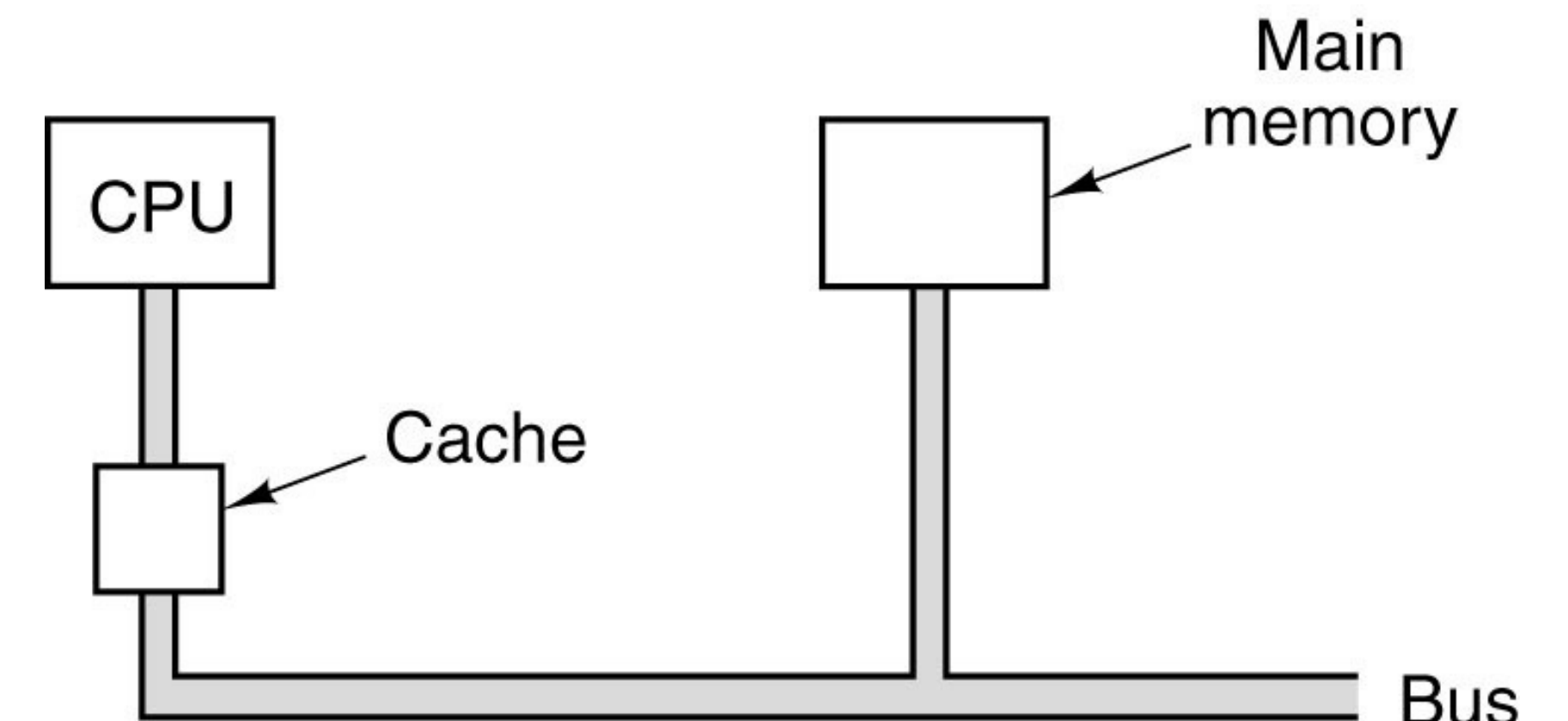
Cache is small to limit cost

Note that "cache" means "hiding place" - transparent to programmer



What is a Cache?

execute typical instruction	1/1,000,000,000 sec = 1 nanosec
fetch from L1 cache memory	0.5 nanosec
branch misprediction	5 nanosec
fetch from L2 cache memory	7 nanosec
Mutex lock/unlock	25 nanosec
fetch from main memory	100 nanosec
send 2K bytes over 1Gbps network	20,000 nanosec
read 1MB sequentially from memory	250,000 nanosec
fetch from new disk location (seek)	8,000,000 nanosec
read 1MB sequentially from disk	20,000,000 nanosec
send packet US to Europe and back	150 milliseconds = 150,000,000 nanosec

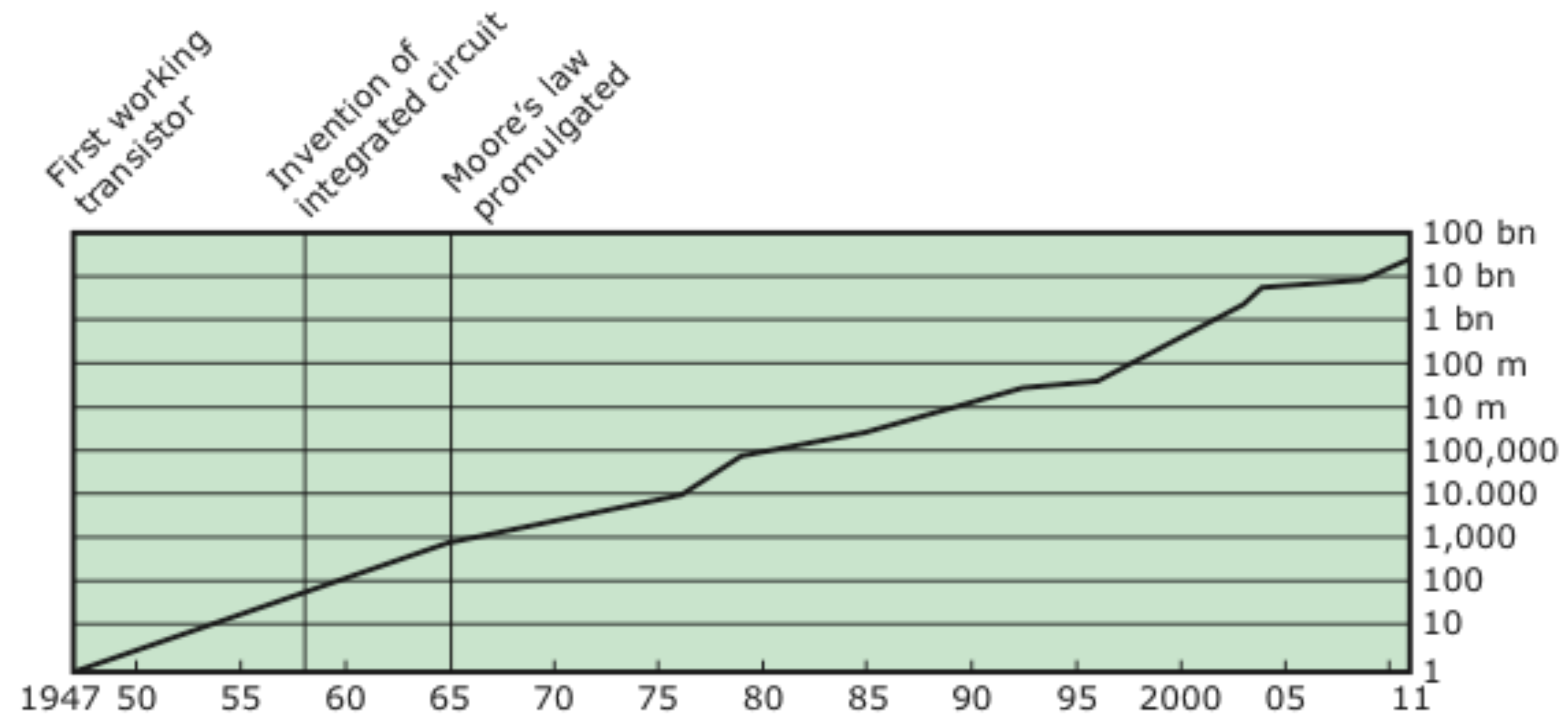


Moore's Law

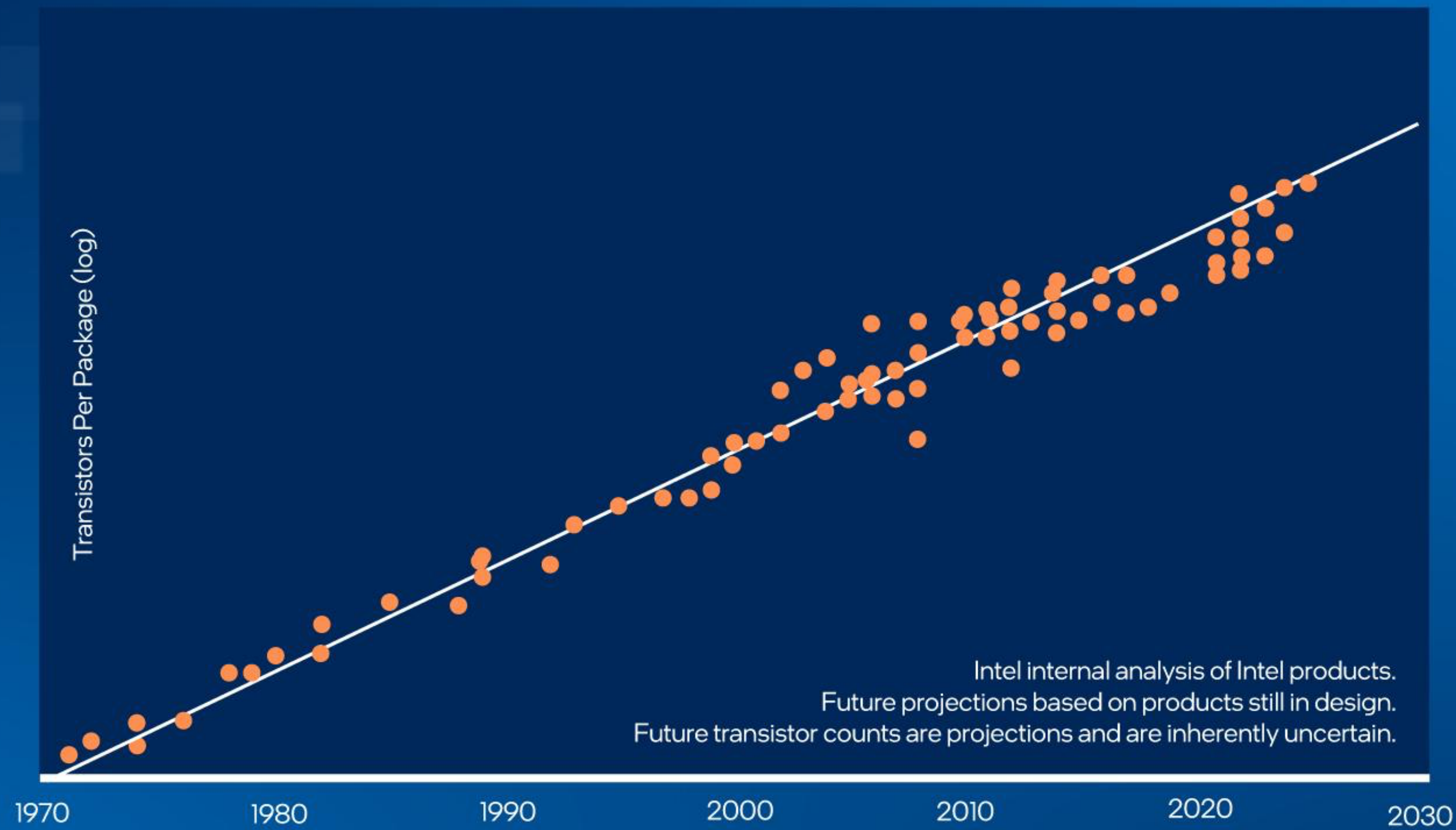
"The complexity for minimum component costs has increased at a rate of roughly a factor of two per year. Certainly over the short term this rate can be expected to continue, if not to increase. Over the longer term, the rate of increase is a bit more uncertain, although there is no reason to believe it will not remain nearly constant for at least 10 years." - **Gordon Moore (co-founder of Intel), *Electronics*, Volume 38, No. 8, April 1965**

Now more commonly expressed as "The number of transistors on a memory chip doubles every 18 months"

Moore's Law



Reproduced from: W.Stallings, "Computer Organization and Architecture", 9th Edition, Pearson, 2013



Aspiring to
1 Trillion
transistors in 2030

- ✓ RibbonFET
- ✓ PowerVia
- ✓ High NA
- ✓ 2.5D/3D packaging

intel®

Moore's Law

The cost of computer logic and memory circuitry has fallen at a dramatic rate

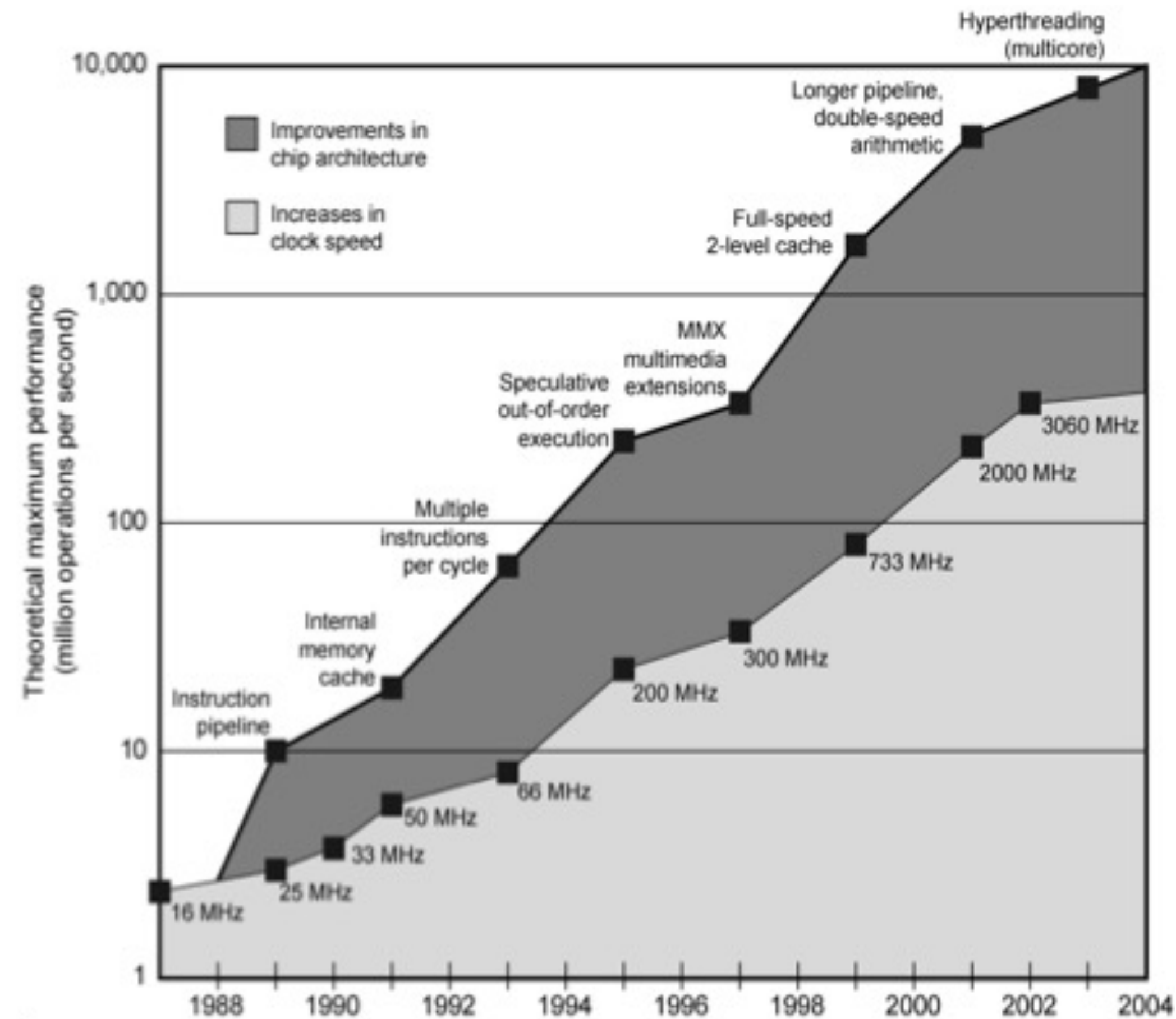
As logic and memory is placed closer together on more densely packed chips, the electrical path is shortened, increasing operating speed

Computers have become smaller, making them more convenient for use in a variety of environments, especially embedded systems

Reduction in power and cooling requirements

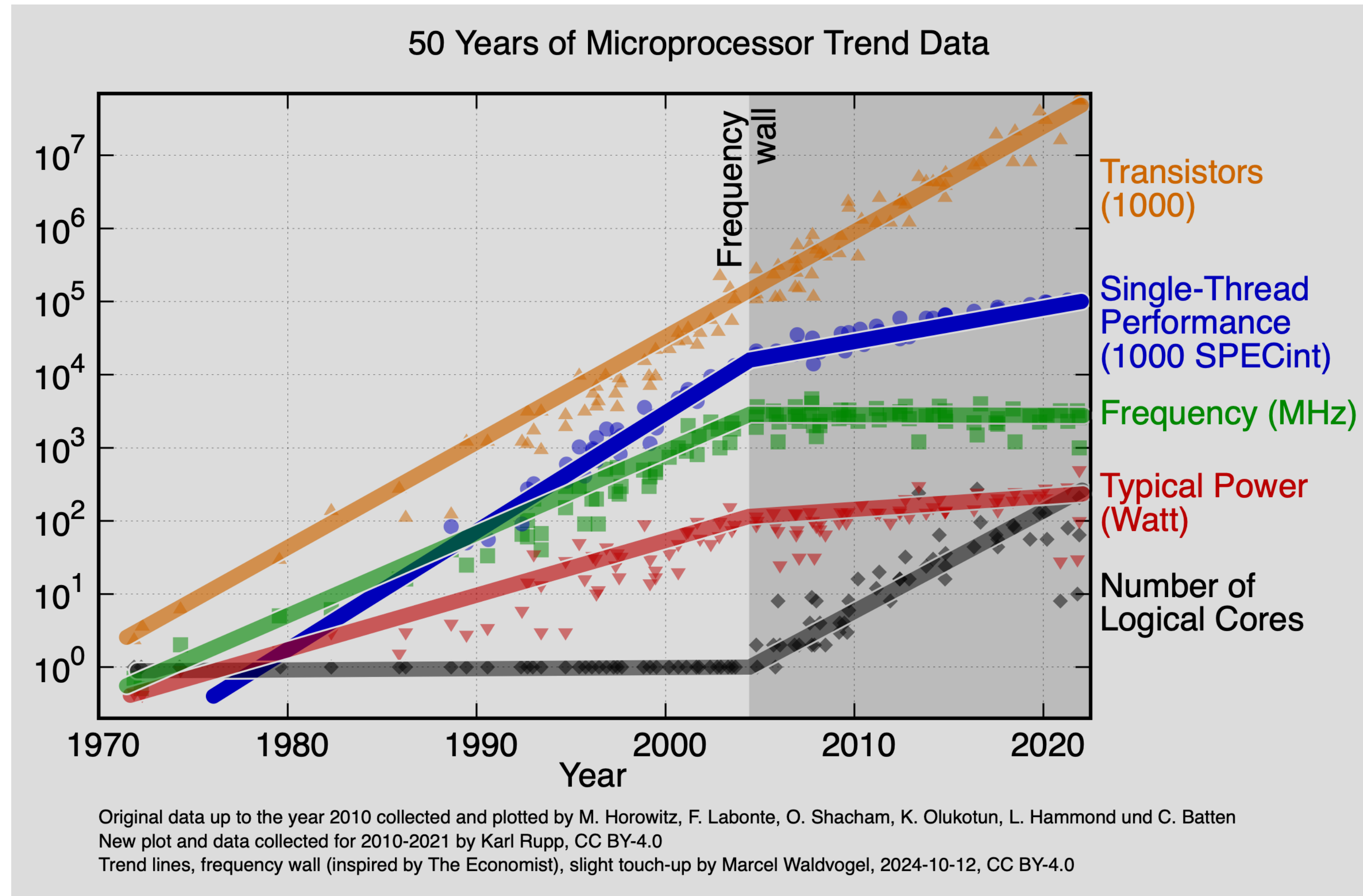
Interconnections on an IC are more reliable than solder connections, thus having fewer off-chip connections increases reliability

Intel Microprocessor Performance



Reproduced from: W.Stallings, "Computer Organization and Architecture", 8th Edition, Pearson, 2010

Current Outlook - Hardware



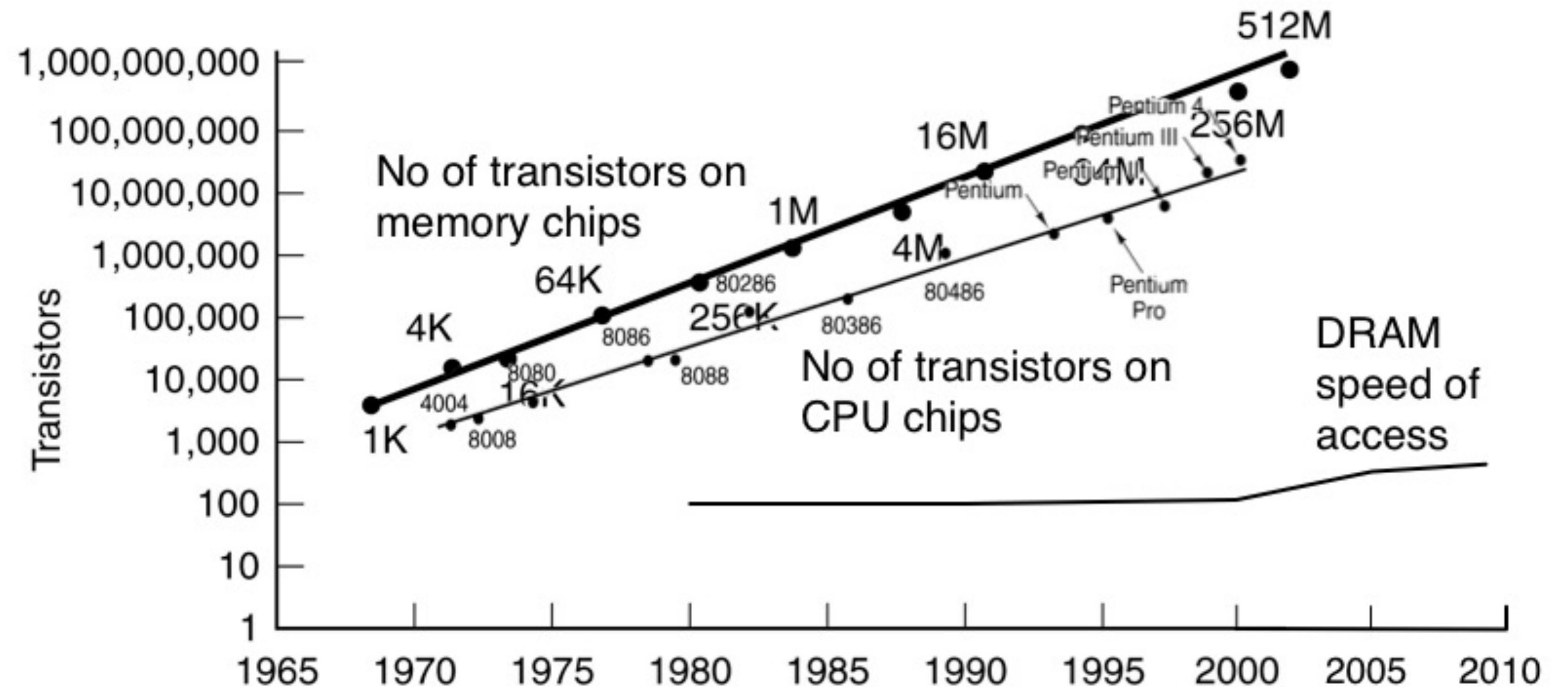
<https://netfuture.ch/2024/10/50-years-of-microprocessor-trend-data/>

Start here 24 Nov

Why Have a Cache?

Memory size is increasing according to Moore's Law

Memory access speed is improving much more slowly



Cache Concepts

Caching read-only data is relatively straightforward

We don't need to consider the possibility that items will change, hence copies across the memory hierarchy will remain consistent

Two general strategies can be adopted when caching writes, both of which can employ a write buffer to enhance performance

Write Through - Updates the item in cache and writes through to update lower levels of the memory hierarchy

Write Back - Only updates copy in cache, ensuring that blocks are copied back to memory when when they are to be replaced

Types of Cache Miss

Measures of cache performance include hit rate (h) and miss rate ($1-h$)

$$h = \frac{\text{Number of times the words are in cache}}{\text{Total number of memory references}}$$

We can understand cache misses by sorting all misses into categories

Compulsory - Misses that would occur regardless of cache size, e.g., the first access to a block can not be in cache (regardless of cache size) so the block must be retrieved

Capacity - Misses occurring because a cache is not large enough to contain all blocks needed during the execution of a program

Conflict - Misses occurring as a result of the placement strategy for blocks not being fully associative, which means that a block may be discarded and retrieved

Coherency - Misses occurring due to cache flushes in multiprocessor systems

Measuring Cache Performance

Performance measures based solely on hit / miss rates don't factor in the cost of a cache miss - the real performance issue!

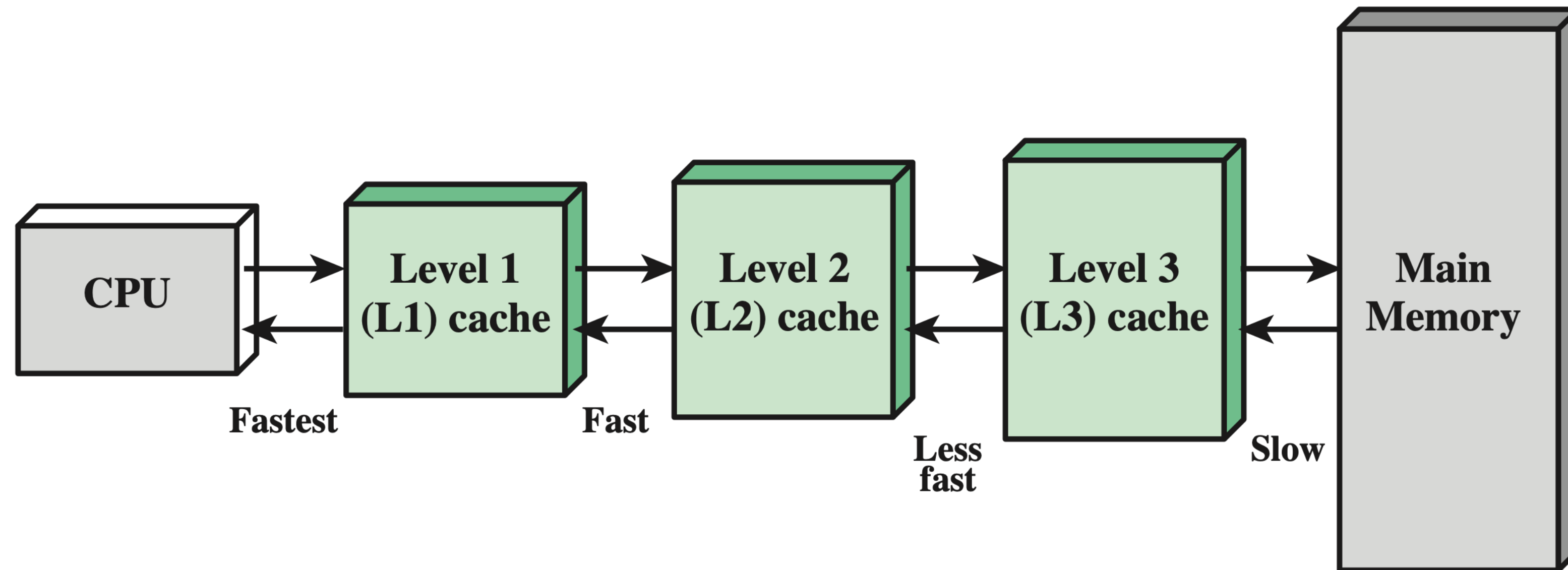
Average memory access time is an alternative measure that accounts for the cost of a cache miss

$$\text{Average memory access time} = \text{Hit time} + (\text{Miss rate} \times \text{Miss Penalty})$$

This is where hit time is the time to hit in the cache and the miss penalty is the time taken to replace the block from memory

Even average access time is still an indirect measure of performance - execution time can often be a better measure in real-world situations

Multilevel Cache



Reproduced from: W.Stallings, "Computer Organization and Architecture", 9th Edition, Pearson, 2013

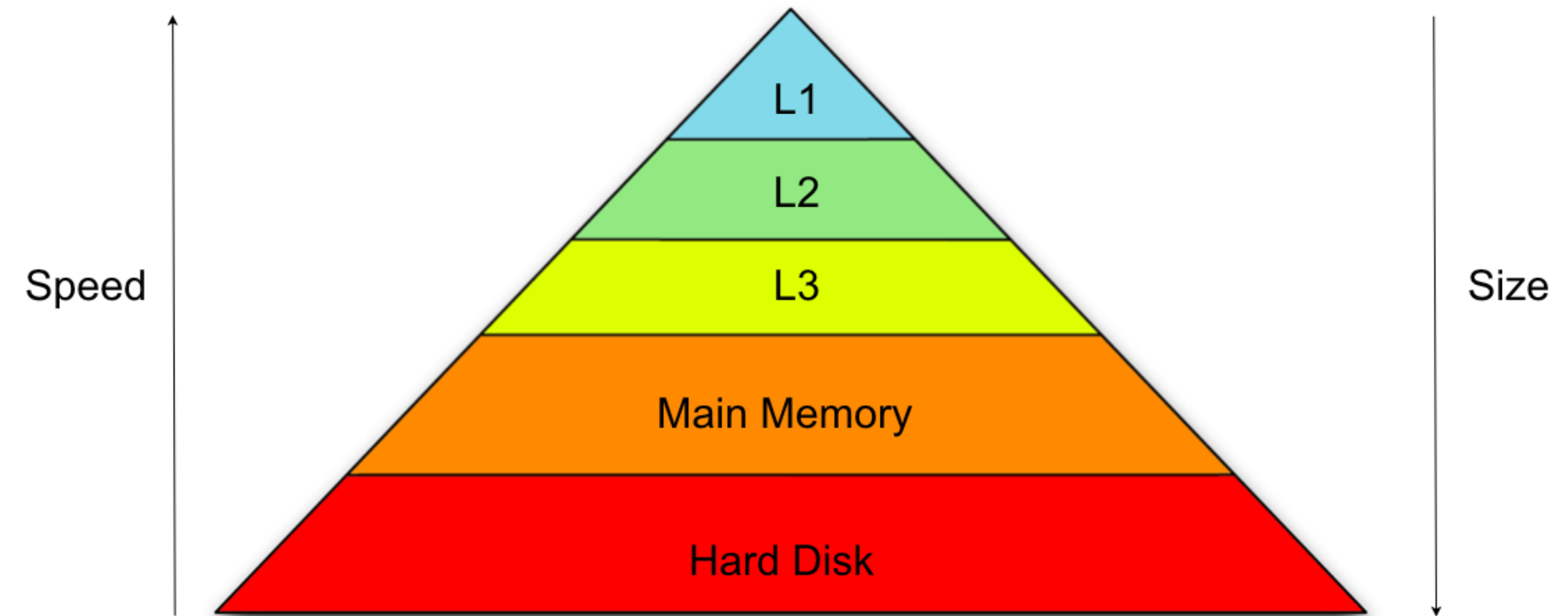
Relating the Memory Hierarchy to Cache

Data storage is a balance of size and speed:

Memory closer to the CPU (Cache - L1, L2, L3) is typically faster but has a far smaller capacity.

Main memory is further and slower but has a far greater capacity

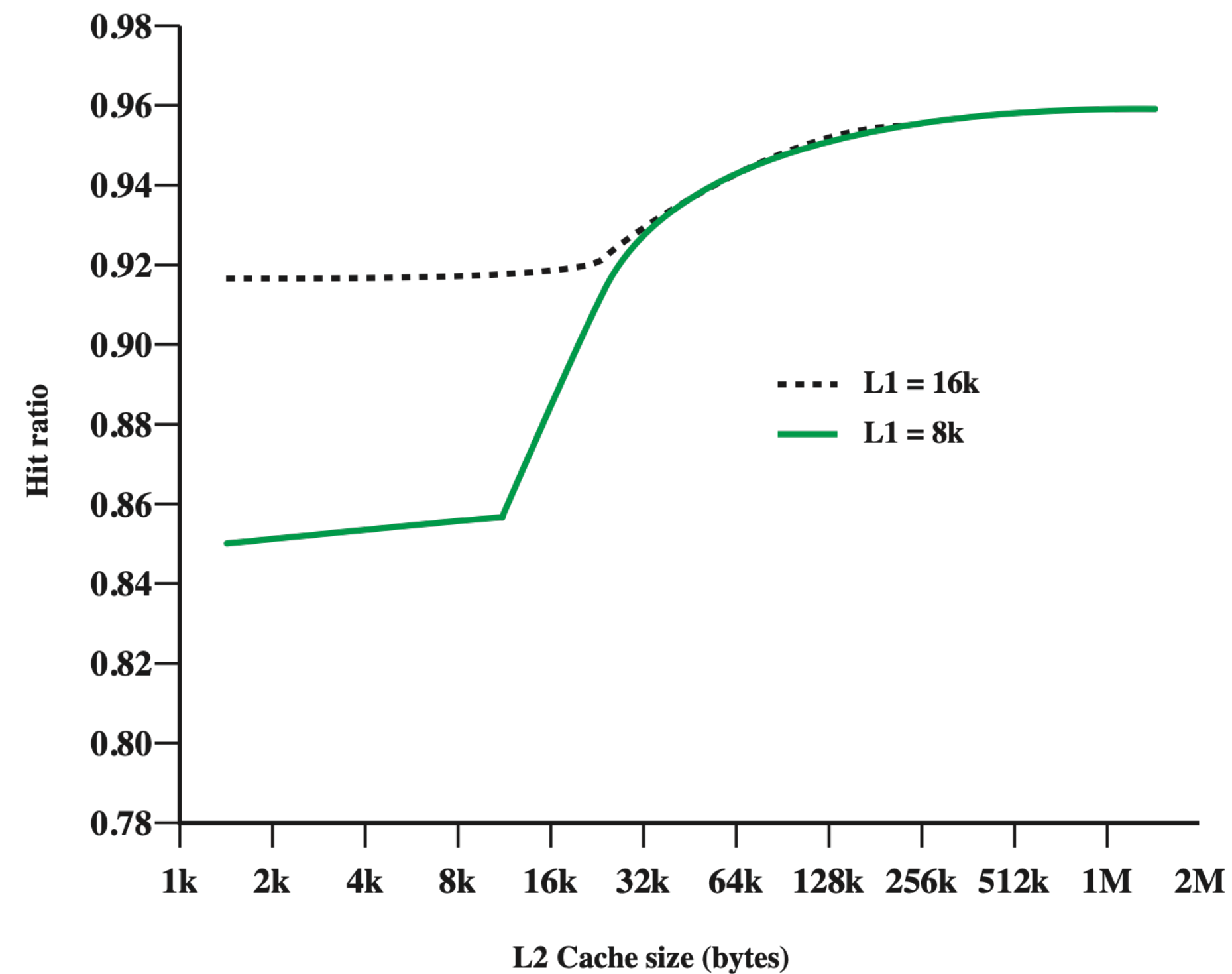
Hard disks have the greatest capacity, and do not need continuous power, but have the slowest access times.



Optimisation: We want to use fastest memory possible for high performance, but problems may large datasets.

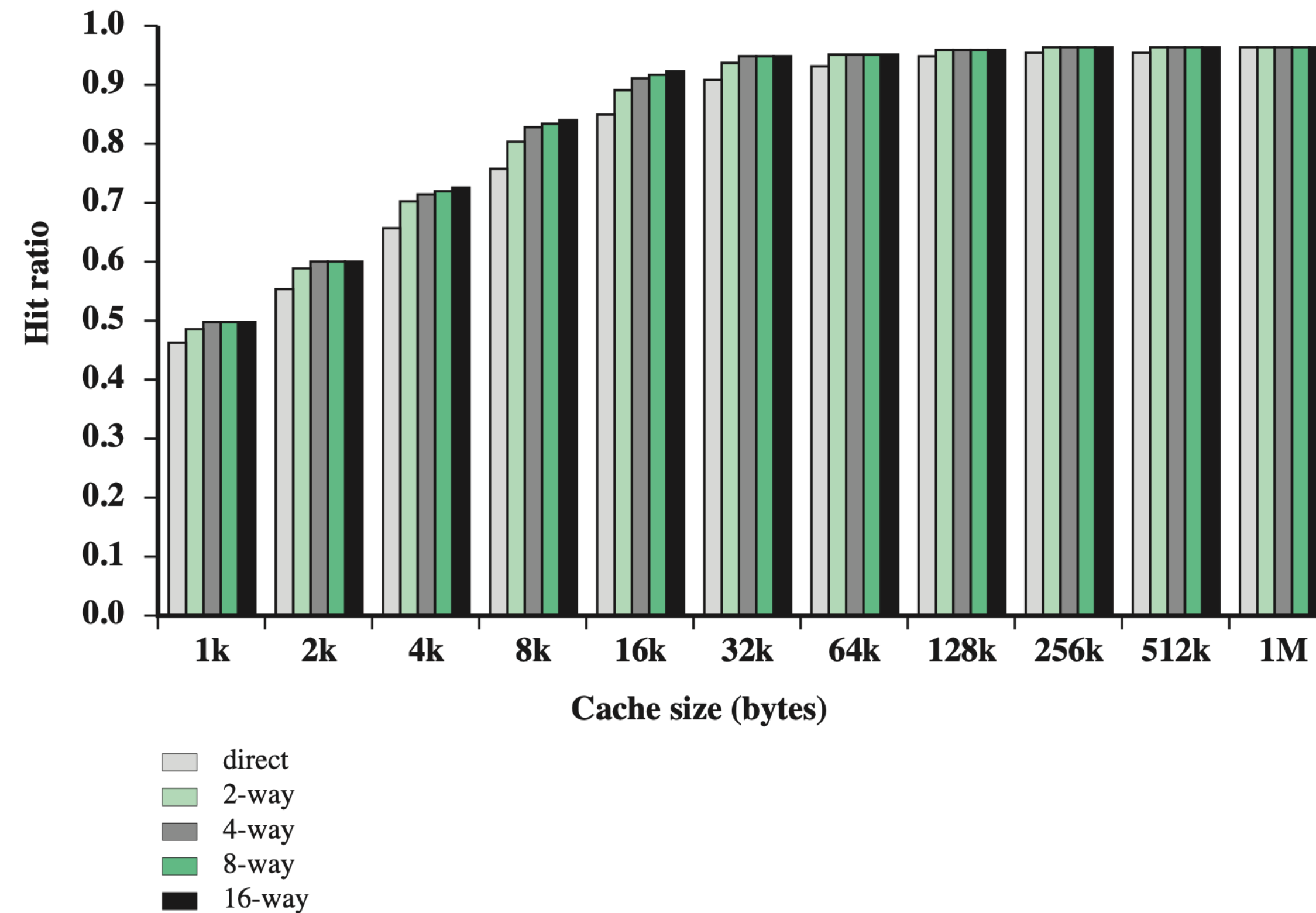
Solution: Reuse data already in cache as much as possible and prefetch data from main memory into cache before it is needed, masking load times

L1 Cache Size and Complexity



Reproduced from: W.Stallings, "Computer Organization and Architecture", 9th Edition, Pearson, 2013

L1 Cache Size and Complexity



Reproduced from: W.Stallings, "Computer Organization and Architecture", 9th Edition, Pearson, 2013

Memory Cell Organisation

Semiconductor Memory

Most common form of main store

RAM (Random Access Memory)

Two main technologies

- Static RAM (SRAM)

- Dynamic RAM (DRAM)

SRAM use a flip-flop as storage element for each bit

DRAM for each bit, use the presence or absence of charge in a capacitor to denote a 1 or 0

Capacitor charge leaks away over time - requires periodic refreshing (more complex) but is cheaper than SRAM so is more commonly used

Static RAM (SRAM)

SRAM stores data using configurations of flip-flops and logic gates

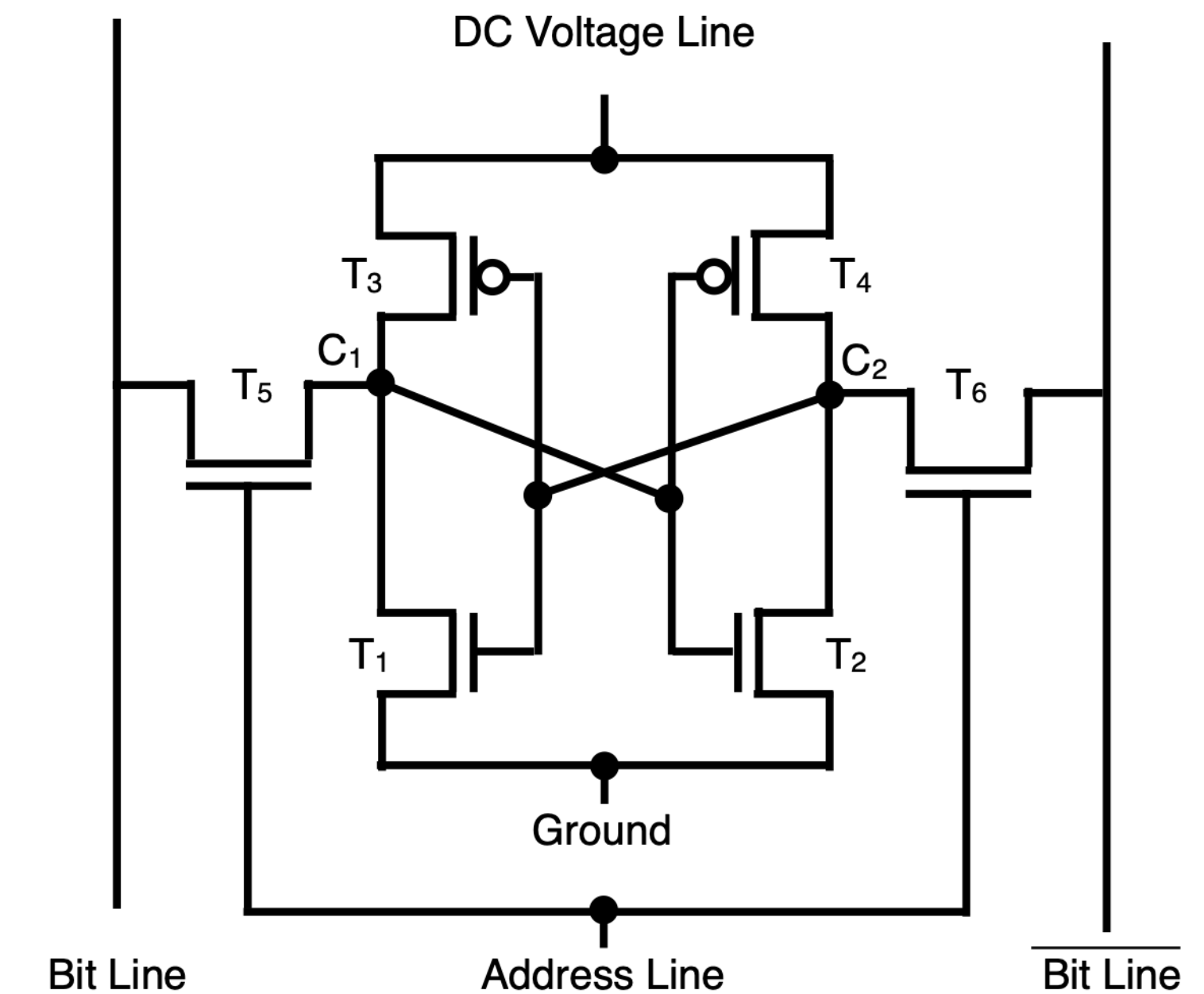
Memory cells hold data as long as power is supplied

Typical SRAM cell uses four cross-coupled transistors to establish a stable logic state

When storing logical 1, C1 is high and C2 is low (T1 and T4 off, T2 and T3 on)

When storing logical 0, C1 is low and C2 is high (T1 and T4 on, T2 and T3 off)

Address line controls two transistors (T5 and T6), where both transistors being on permits read and write



Write operation by applying bit value to Bit Line, which sets T1, T2, T3 and T4 to the appropriate bit value

Read operation by reading Bit Line

Dynamic RAM (DRAM)

DRAM memory cells store data as charge on capacitors

Capacitors naturally discharge, hence DRAM cells require periodic charge to maintain data storage

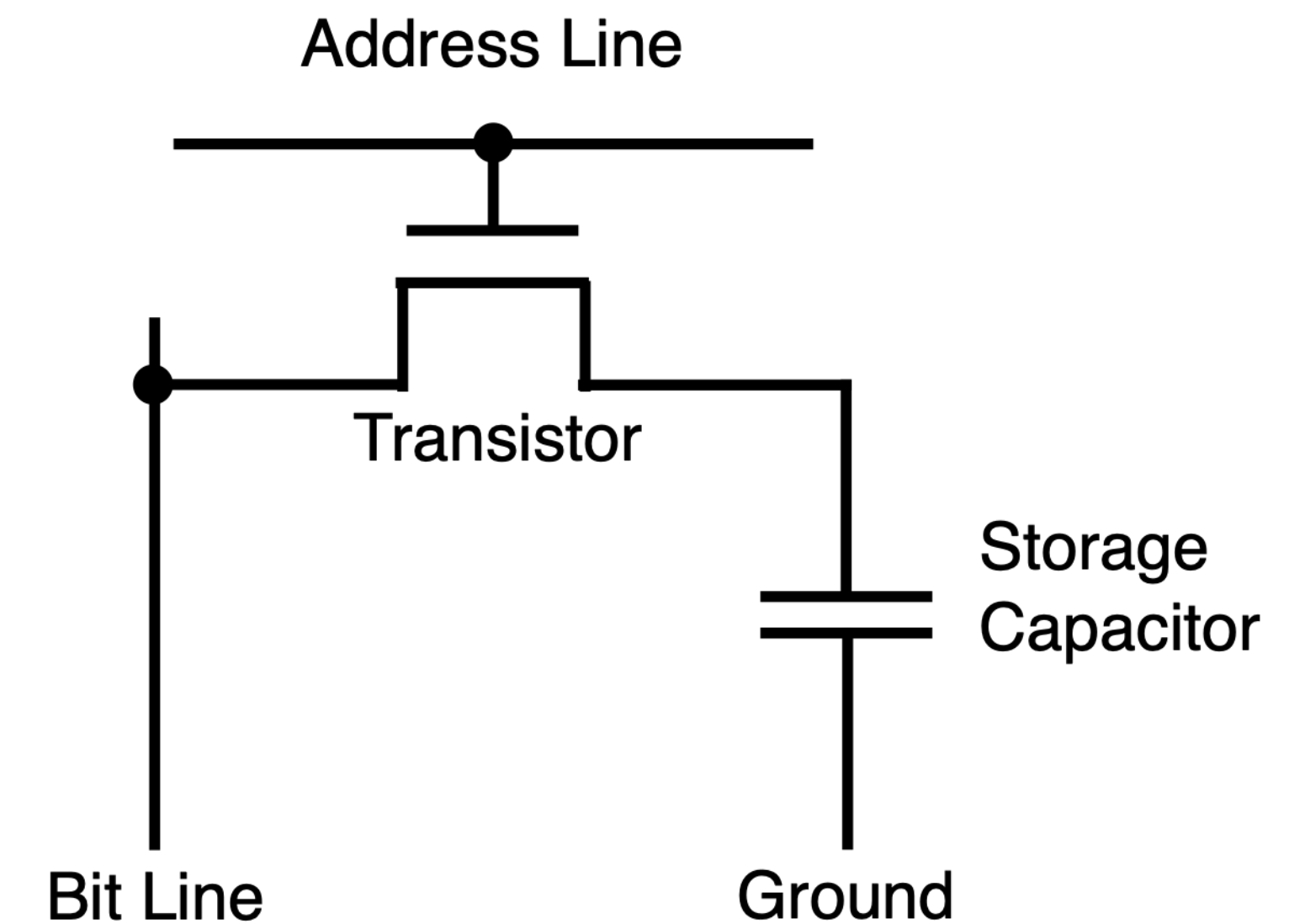
The term 'dynamic' refers to the natural leakage of charge that takes place even when power is applied

Presence or absence of charge is interpreted as a logical 0 or 1

We can appreciate the operation of DRAM by considering a 1-bit DRAM memory cell

Address line is activated when the bit value from the cell is to be read or written

A transistor acts as a switch that is closed (allowing current flow) if a voltage is applied to the address line and open if a voltage is not applied to the address line (no flow)



DRAM Operation

Write operation:

Voltage applied to the bit line (high for logical 1, low for logical 0)

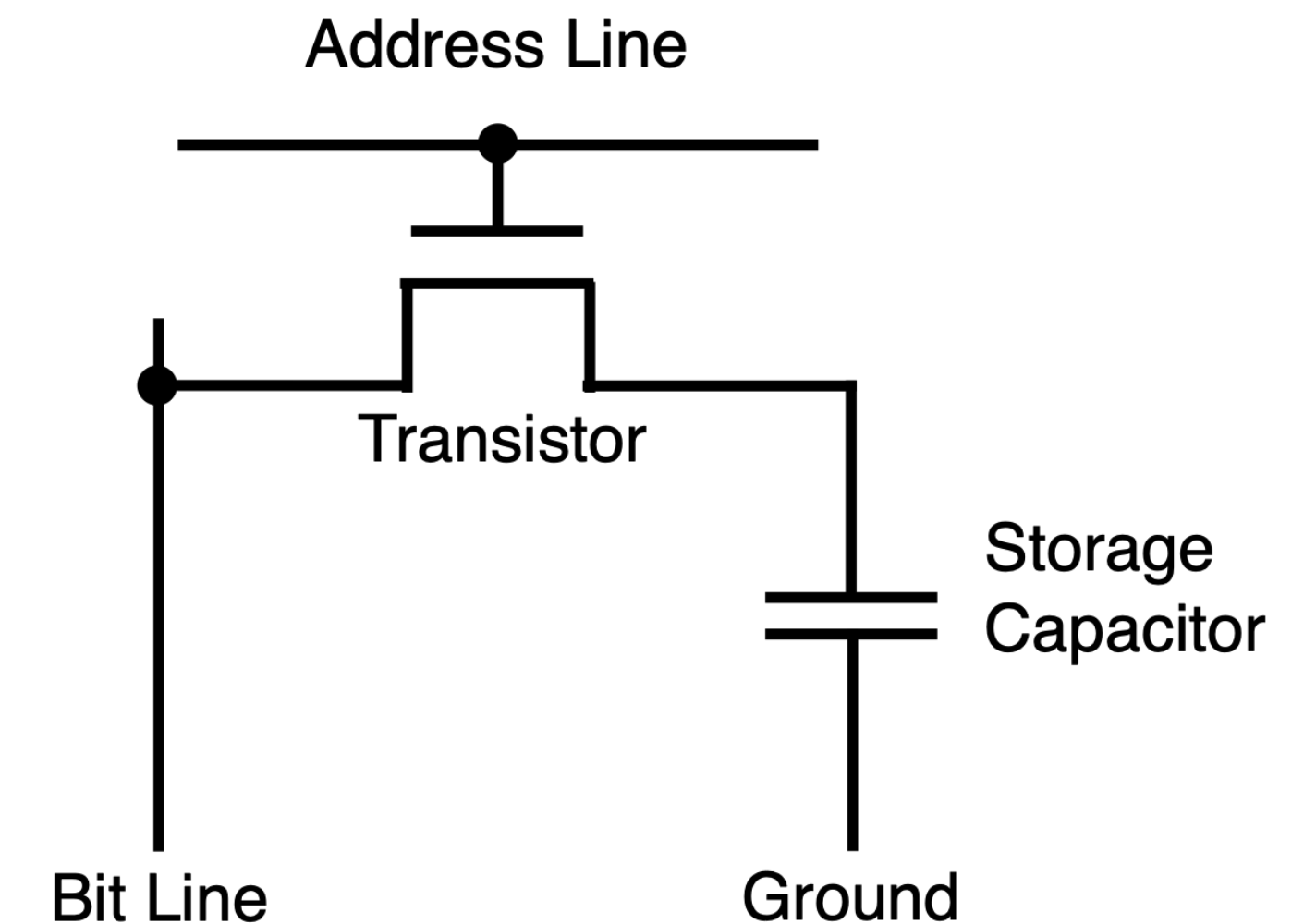
A signal is then applied to the address line, allow a charge to be transferred to the capacitor (data store)

Read operation:

Transistor turns on when address line is selected, allowing the stored charge to be fed out to the bit line and a sense amplifier

Sense amplifier compared the capacitor voltage to a predetermined threshold value to determine if the cell contains a logic 0 or 1

The read process discharges the capacitor, which means that it must be restored (as well as refreshed) following the read operation



Comparing SRAM and DRAM

Both are volatile - power must be continuously applied

Dynamic memory cells are generally simpler and more compact

- Allows greater memory cell density (more cells per unit area)

- Cheaper to produce than equivalent SRAM memory

Refresh circuitry incurs one-off cost that is only compensated for by larger memory capacities

SRAM are typically provide better read and write times than DRAM

Cache memory (on and off chip) is often implemented as SRAM, whilst DRAM is commonly used for main memory

Advanced DRAM Organisations

The interface to main memory is a critical performance bottleneck

Motivation for multiple high-speed SRAM caches between a processor and DRAM main memory

To achieve better system performance we can also consider advanced DRAM organisations

Synchronous DRAM (SDRAM)

Rambus DRAM (RDRAM)

DDR SDRAM

Cache DRAM (CDRAM)

Memory Organisation

Basic element is a memory cell

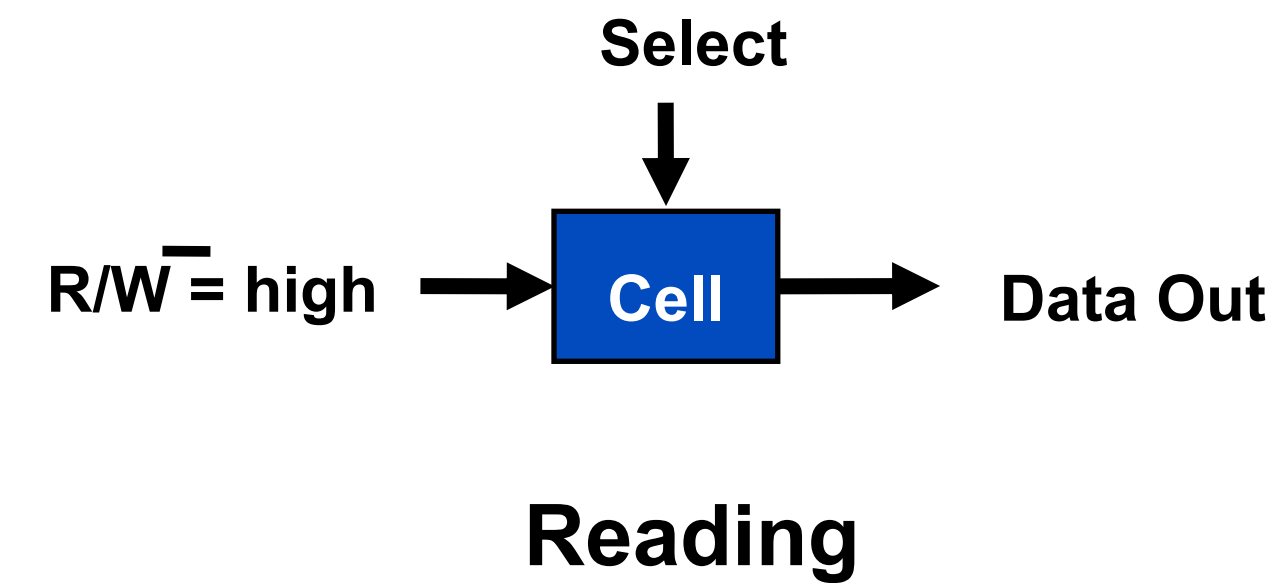
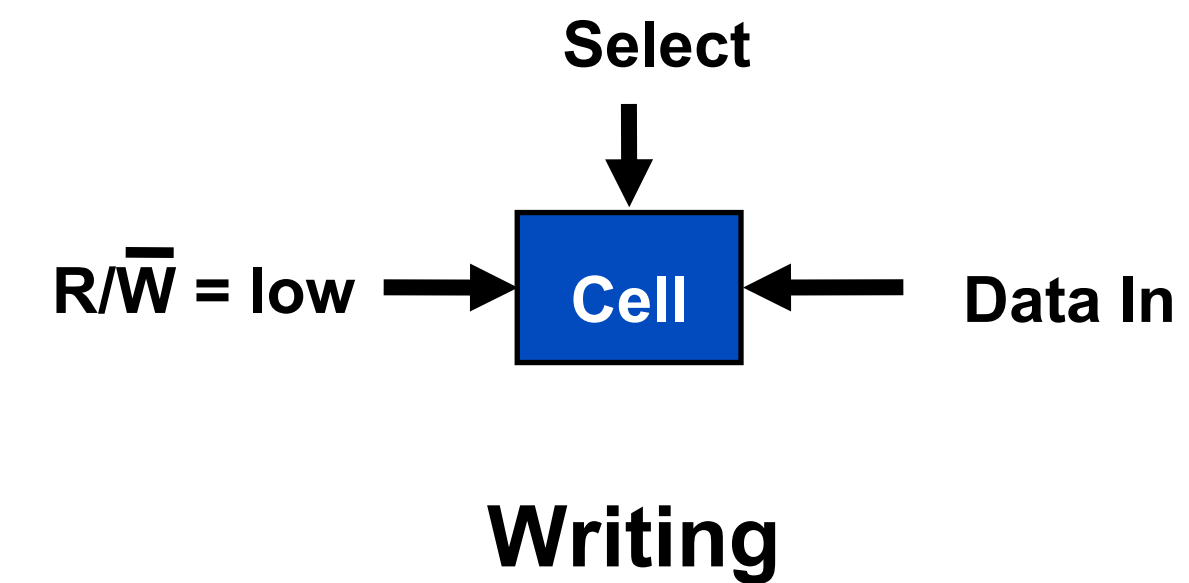
Two states 1, 0

Capable of being written and read

Select – Enable a particular memory cell

R/W – Indicate read or write

Data – Input when writing or output when reading

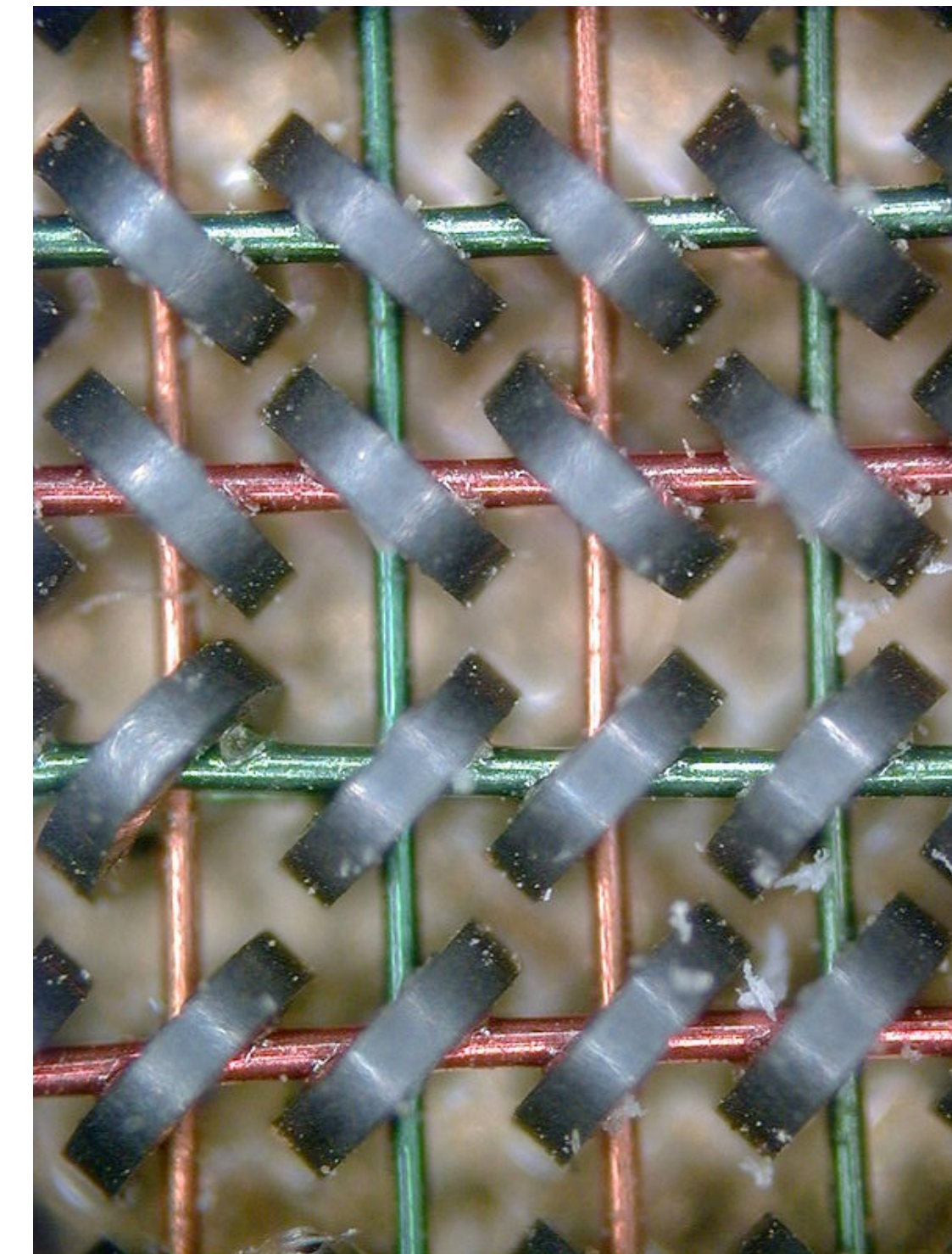


Core Memory

An early form of RAM where data is stored through the magnetisation of tiny metal rings - one bit per ring

Rings are organised into a grid array

Individual bits are accessed by energising row and column wires

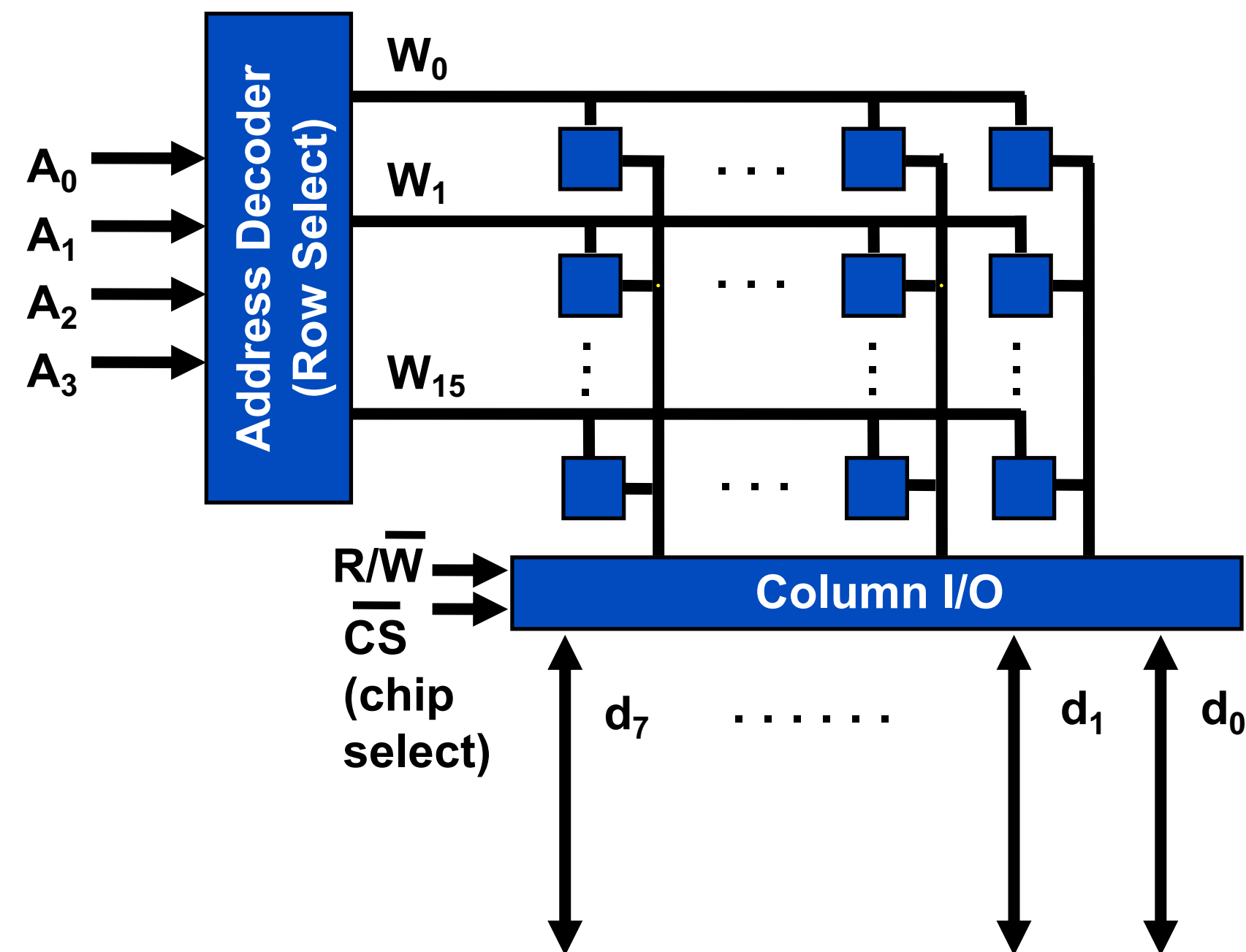


Organisation of a Memory Chip

As you might expect, it is critical to maintain a structured approach to the organisation of on-chip memory

Example of a 16x8
memory integrated
circuit

16x8 denotes
W x B, i.e., words x bits
16 words, each of 8 bits



Choices in Memory Chip Organisation

IC design for $16 \times 8 = 128$ cells required 4 address inputs ($= \log_2 W$, where W = no of words) and 8 data lines ($= B$, where B = word size)

Consider a 1Kbit = 1024 cells device

Possible to organise as a 128×8 memory cell array

Requires 7 address pins ($\log_2 128$) and 8 data pins

A total of 15 I/O pins, plus power, etc...

Poor Choices in Memory Chip Organisation

Alternative, it is possible to organise as a 1024 x 1 array

Requires 10 address pins ($\log_2 1024$) and 1 data pin

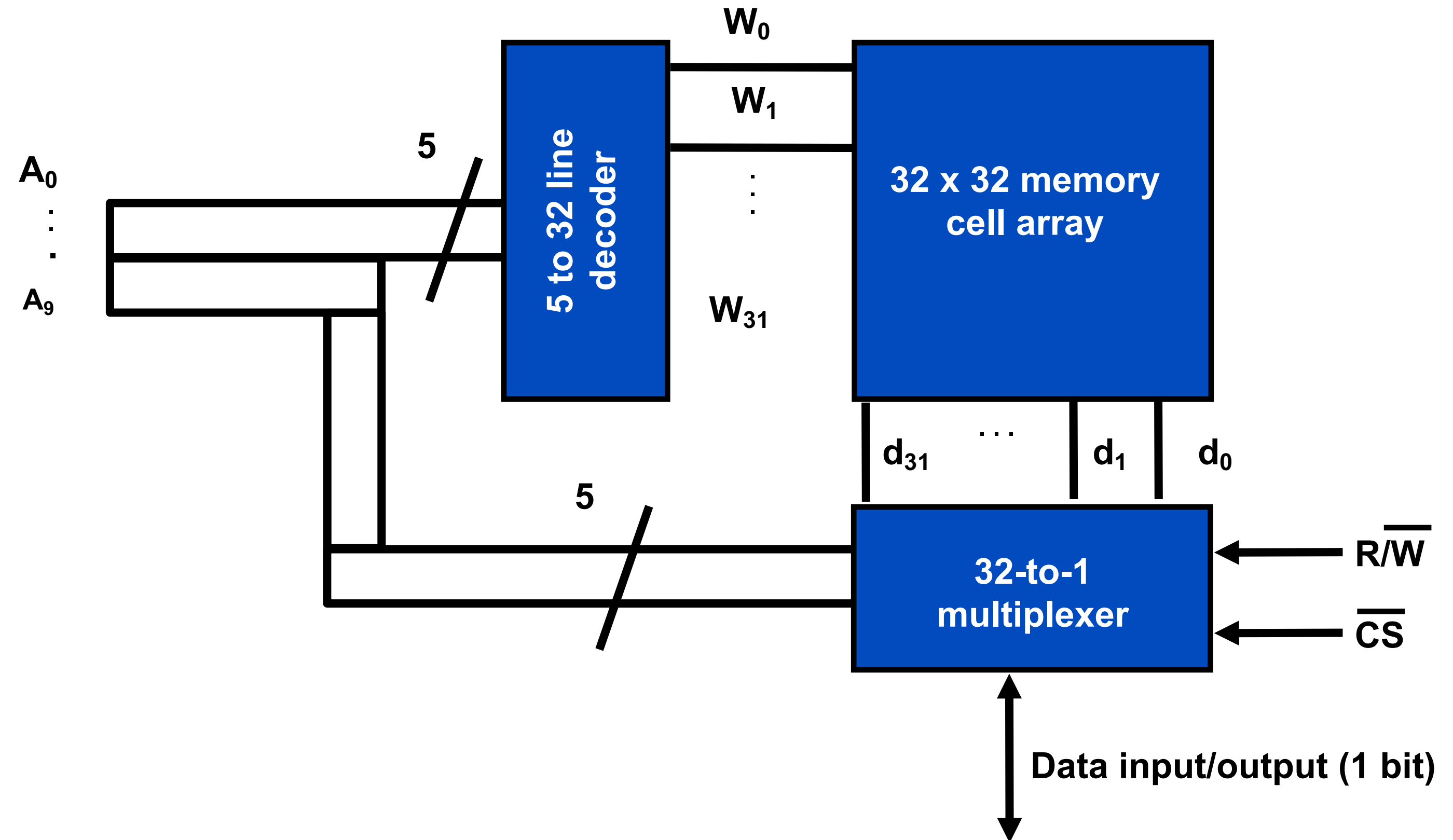
A total of only 11 I/O pins, plus power, etc...

Results in a long, narrow cell array with a large address decoder, which is an inefficient use of space

Minimising Address Decoding Space

Divide the address inputs into 2 parts - row address and column address

This minimises the space required for address decoding and hence maximises space used for memory cells



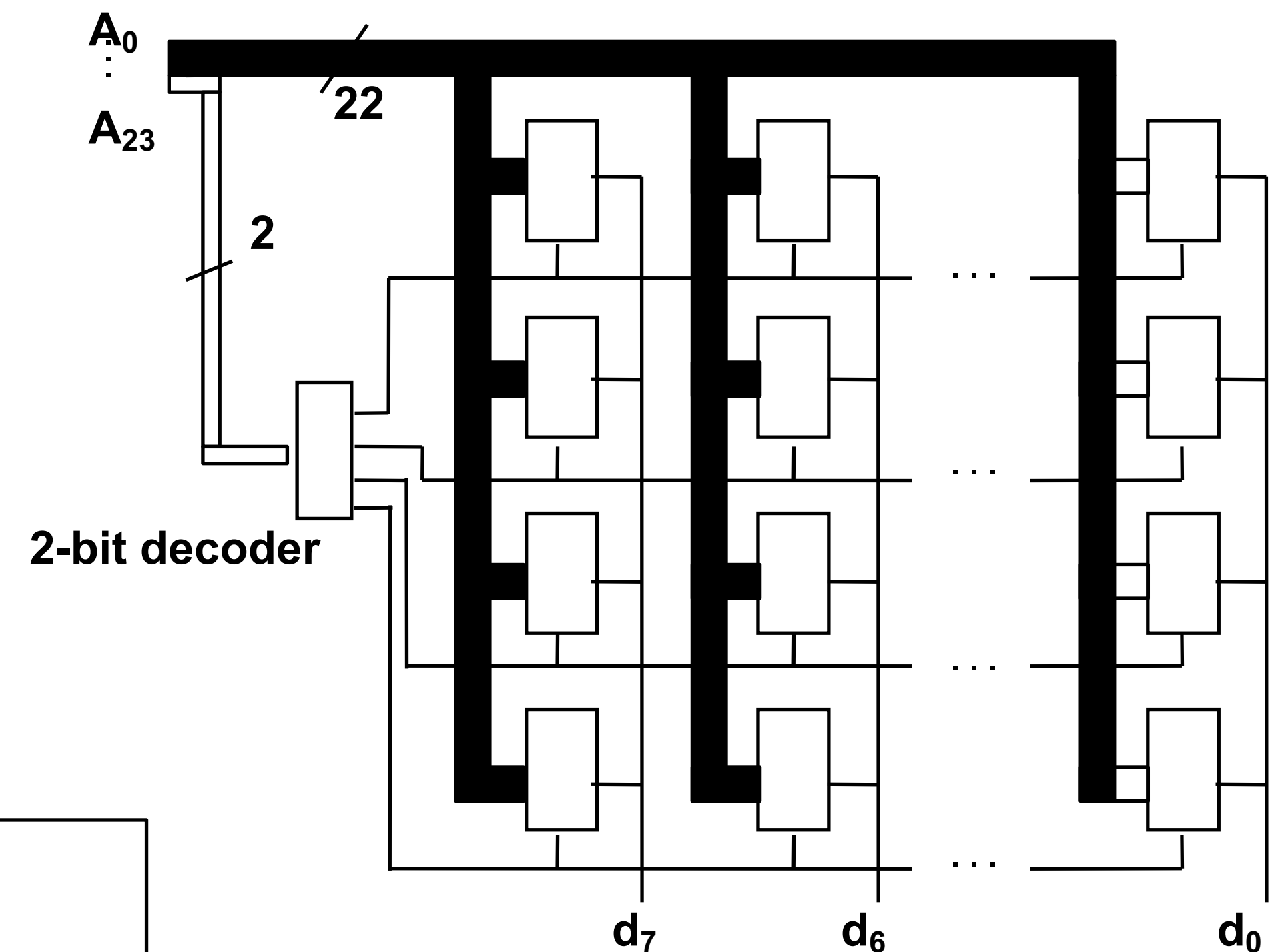
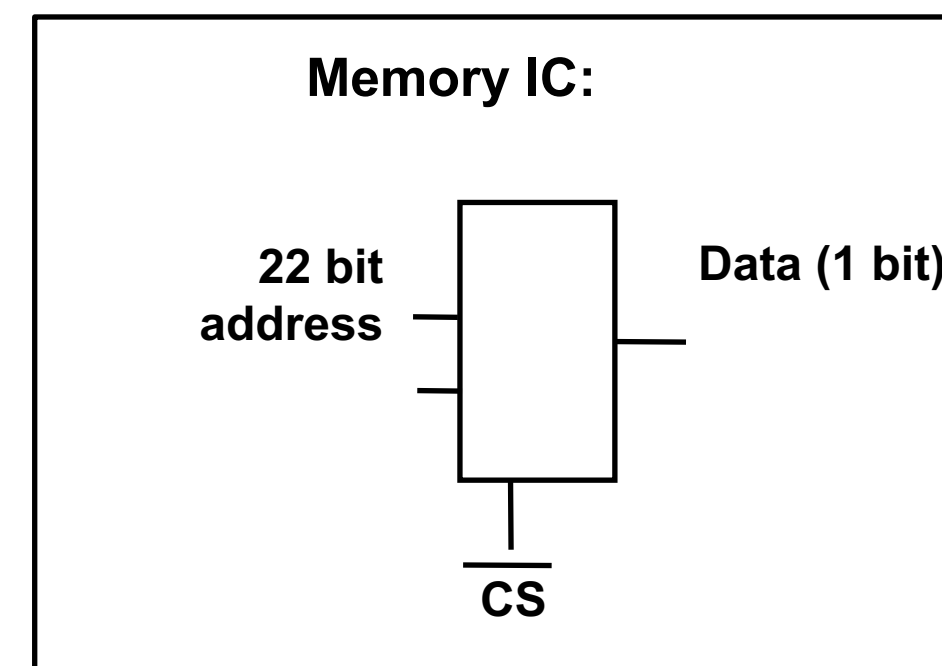
System Memory Organisation

We can use the same idea to organise multiple memory ICs

Here, A_{22} and A_{23} select 1 row of 8 chips, each providing one bit of the 8-bit word

Each chip is 4M x 1 (22 address pins, 1 data pin)

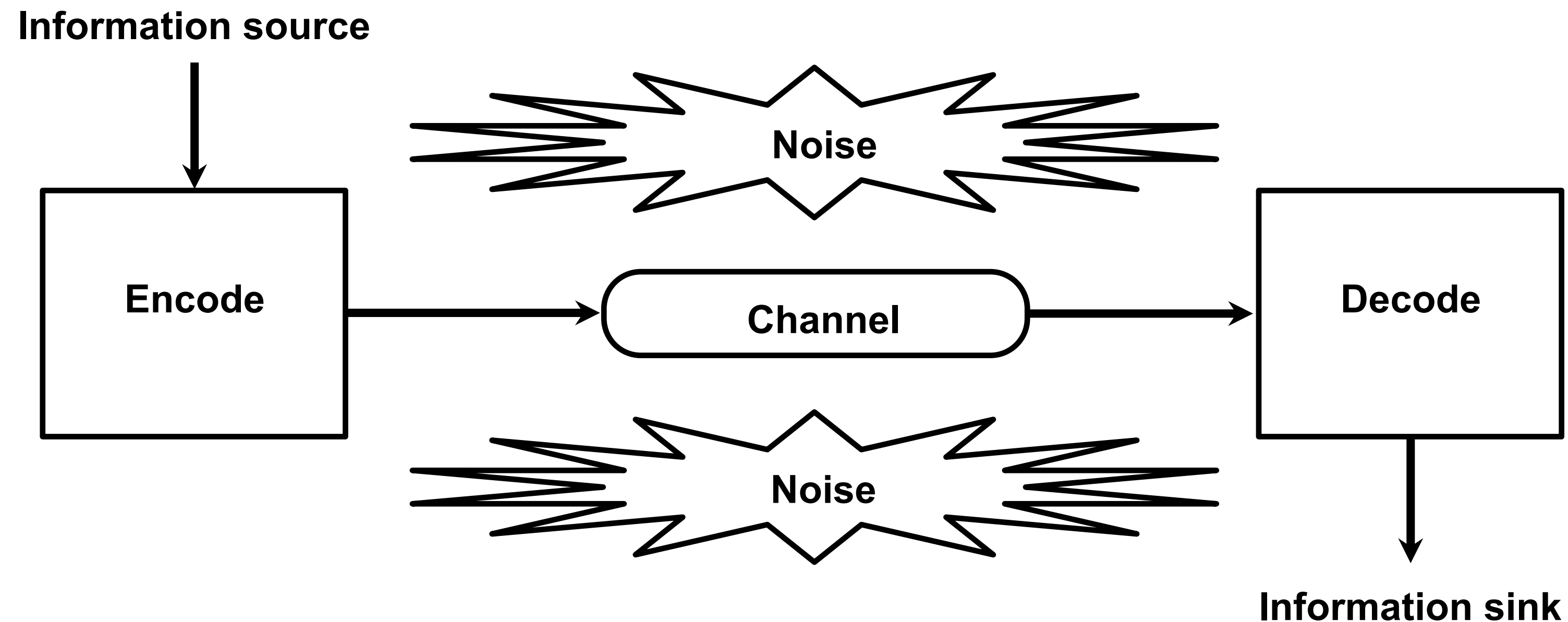
System has 4 rows, 8 chips per row, making the system total a 16M x 8 memory



Detecting and Correcting Errors

Error Detection and Correction

Errors occur within a computer system, e.g., in system memory, and in the communication between systems, e.g., in the transmission of messages



Noise

Noise is unwanted information

It comes in various forms, but is always present, and is one of the limiting factors in computer systems

Noise arises from the physical properties of devices

- Thermal noise

- Noise of electronic components

- Noise of transmission circuits

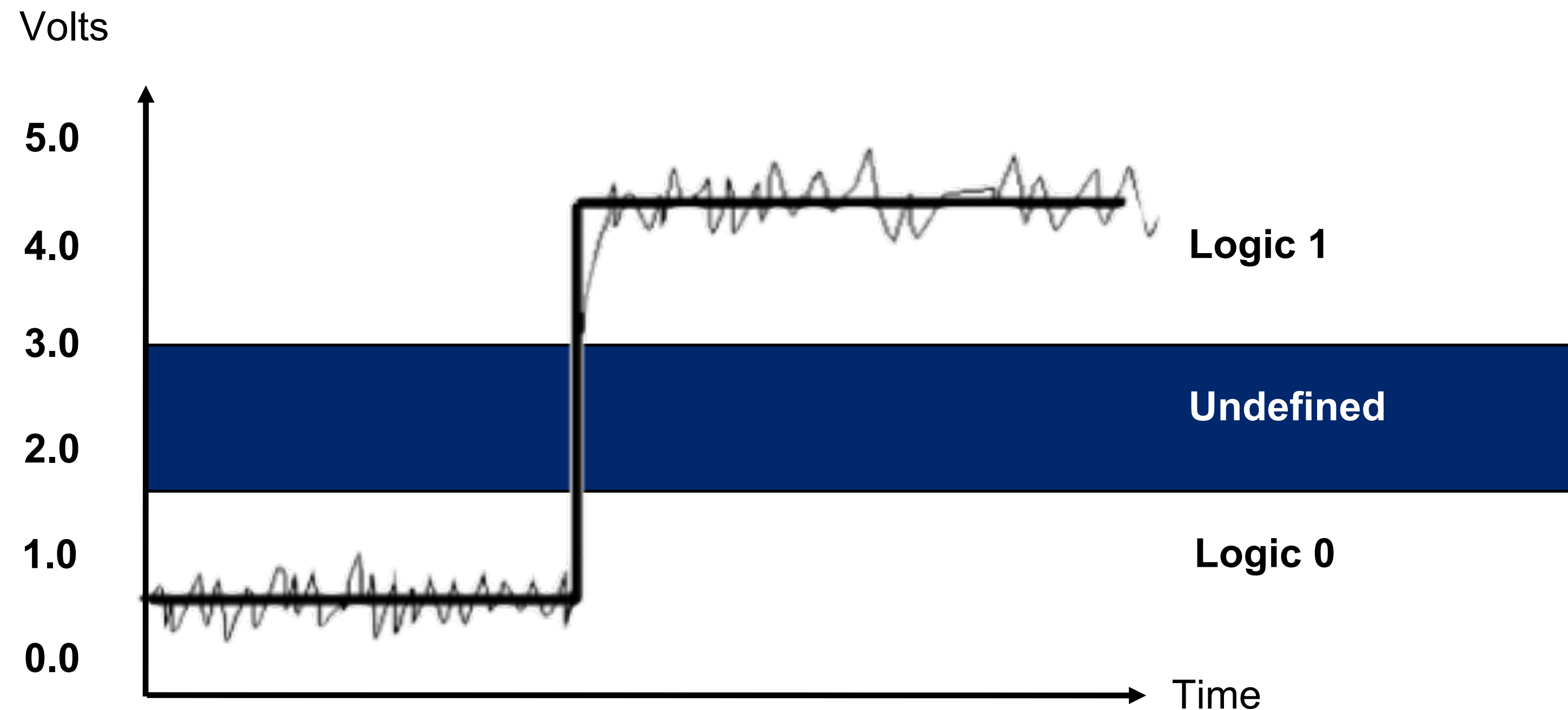
Magnetic media also have a classic form of noise due to random alignment of magnetic fields.

- Decreased area to store a bit means noise gets worse, making errors likely

Noise and Digital Logic Devices

Digital logic gives us a high degree of noise immunity

Immunity collapses once the noise reaches a certain magnitude - our aim is to not let this happen



Detecting Single, Isolated Errors

These are considered to occur at random, usually due to noise

We could send the message three times and take a vote

Very expensive

Observation that, if the probability of an error in the channel is low, the probability of two errors close together is even lower

Thus, we can add a “parity” bit to the message every so often, which “summarises” a property of the message that we can check to determine whether the message has been altered

Much cheaper and adequate in many situations

Parity

Parity adds an extra bit

There are two types of parity system.

Even parity system - The value of the extra bit is chosen to make the total number of logic 1s an even number

Odd parity system - Make the total number of logic 1s odd

Parity bit value can be computed in software or hardware

7-bit ASCII for 'A': A = 0100 0001

Using even parity system: A = 0100 0001

Using odd parity system: A = 1100 0001

Parity in Software

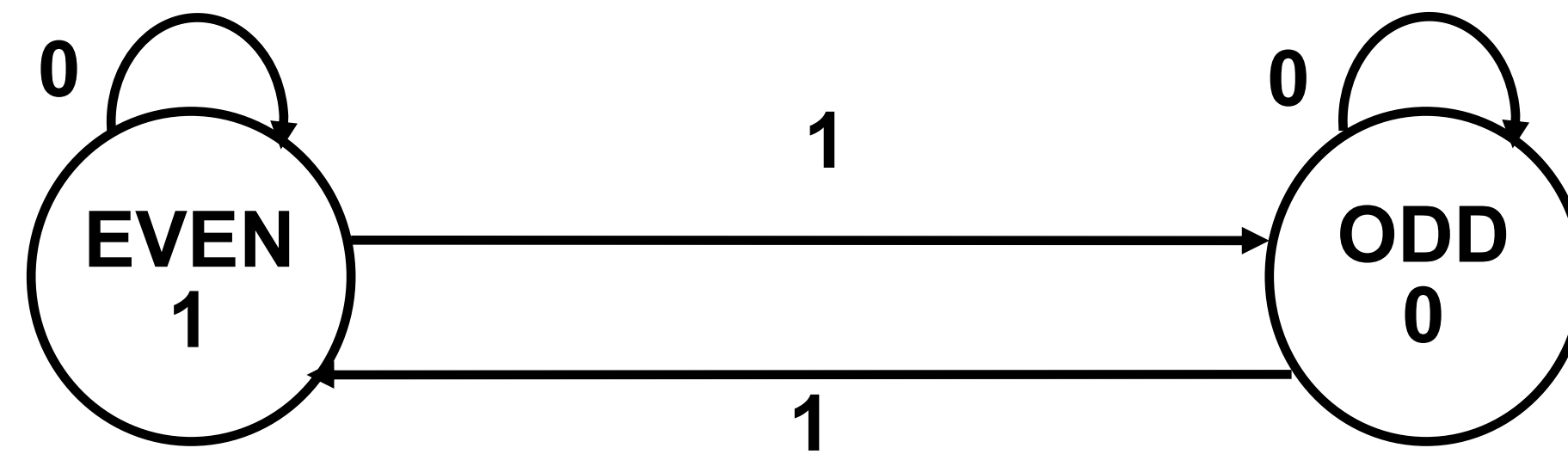
Odd parity system - Start at EVEN

Even parity system - Start at ODD

Each 1 in the message causes a transition of state

Final state at the end of the message is the resulting parity bit

Parity may be computed using a two state Finite Automaton



Parity in Software - Example

For example, try sending 110
using an even parity system

Start with the ODD state, then

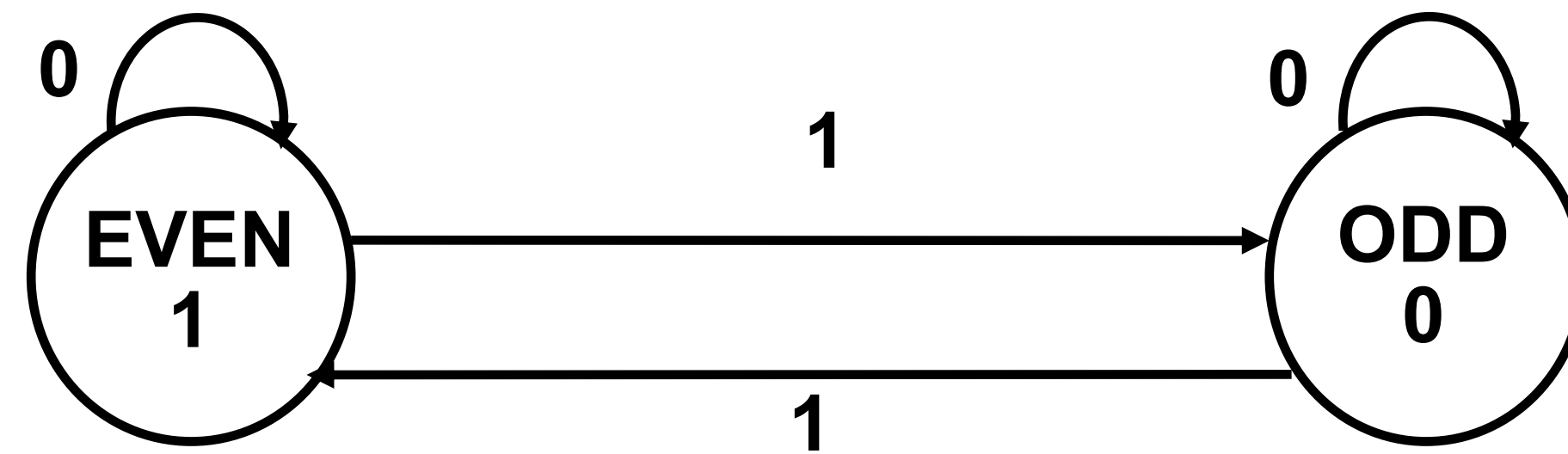
Bit 0 = 0, so go from ODD to ODD

Bit 1 = 1, so go from ODD to EVEN

Bit 2 = 1, so go from EVEN to ODD

Therefore the parity bit is 0 and the message to be sent is 0110

Parity may be computed using a two state Finite Automaton

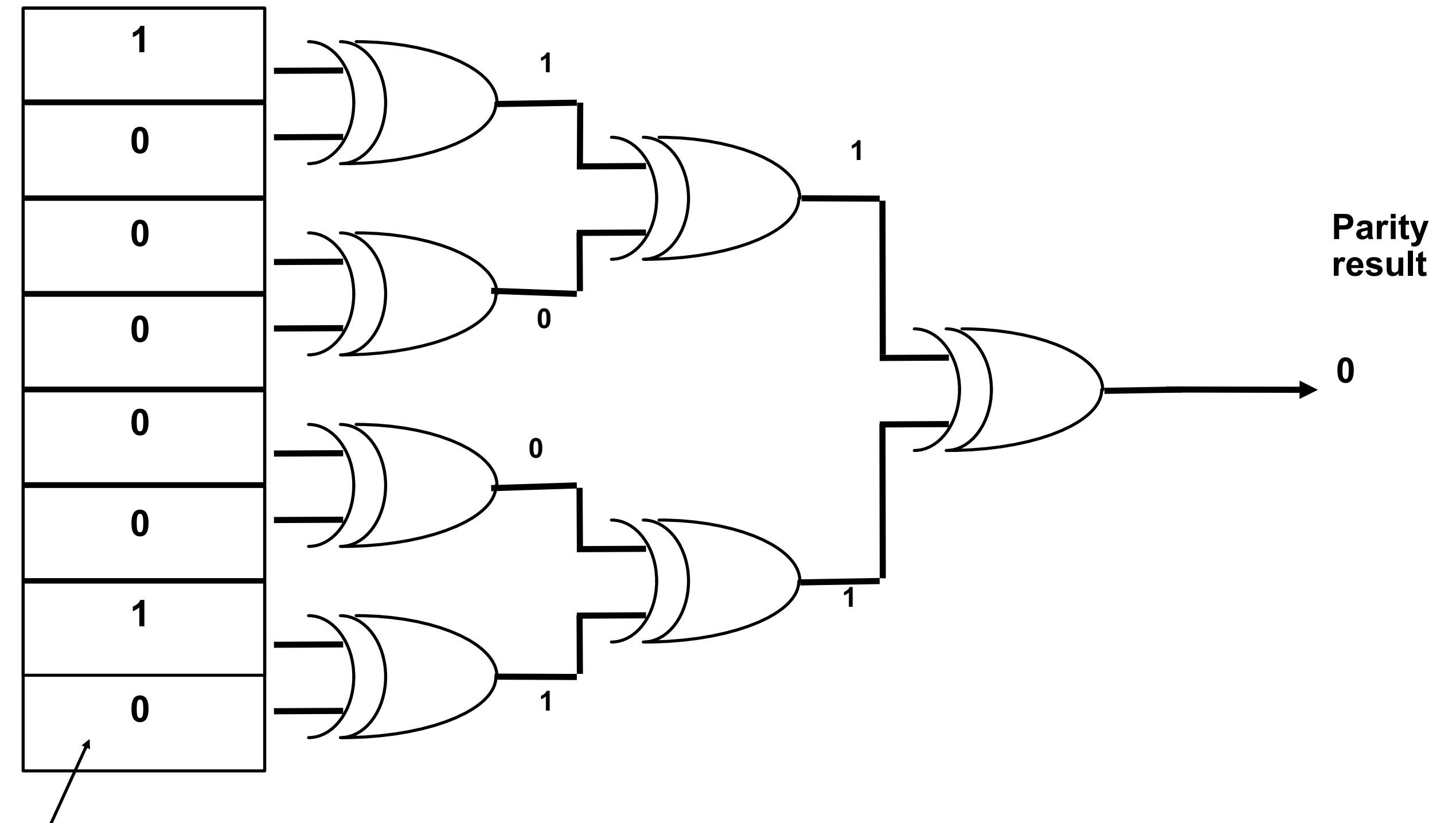


Parity in Hardware

Computing parity in hardware is popular

Hardware implementations generally perform better than software implementations, despite being a relatively simple digital logic circuit

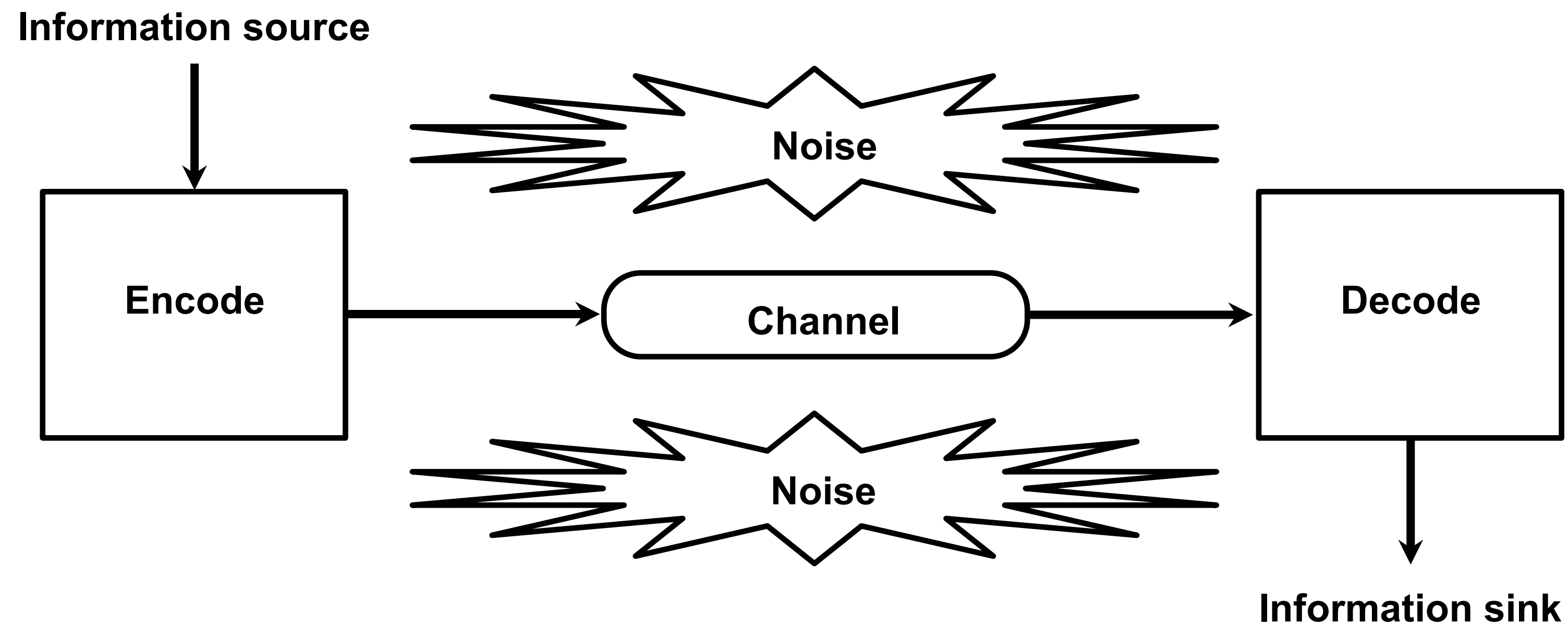
Input:



Dummy input parity bit:
always 0 in even parity system (1 in odd parity system)

Overview of Parity

Parity provides error detection without the need to significantly change our model for communication



Error Detection Using Parity

At our transmitter a message, e.g., 7 bits, is encoded

Compute parity (P) and append parity to message

Send modified message across channel

At our receiver the message, now 8-bits including parity, is decoded

Compute parity (Q) from data bits of received message

Compare computed parity Q with received parity P in message and flag ERROR if these are not consistent

Error Detection Using Parity - Example

Transmitter and receiver have been designed to use even parity

Message: 100 0101

Transmitter computes $P=1$

Transmits: 1 100 0101

An error in the channel changes one bit

Received: 1101 0101. Receiver notes received $P=1$

Receiver uses the data bits (101 0101) to compute $Q=0$

Receiver compares received P and computed Q .

An error has been detected, thus receiver will ask for retransmission

Burst Errors

Remember that, if the probability of an error in the channel is low, then the probability of two errors close together is even lower

In practice, multiple errors often occur in bursts, e.g., a 1ms dropout on a 56k baud modem link can error up to ~58 bits

Worse on broadband, e.g, ADSL

Parity can detect single isolated errors but an even number of errors close together leaves parity unchanged, e.g., 0100 0001 => 0100 1101 => parity unchanged!

The form of parity we have seen is inadequate for burst errors

A possible solution is to compute “checksum” values, i.e., the parity of each bit-column over multiple bytes

Detecting Burst Errors - Example

Checksums vastly improve our error detection capabilities and allow for the detection of burst errors

1. To send the following message over a channel:
(NB 7-bit ASCII without parity)

M	100 1101
e	110 0101
s	111 0011
s	111 0011
a	110 0001
g	110 0111
e	110 0101

2. Calculate bit-column parity "checksum" values,
e.g., using even parity

100 1011 = ASCII 'K'

3. Send "Message**K**" instead of "Message"

This system detects all burst errors < 14 bits
Fails if an even number of errors occurs in a bit-column

Error Correcting Codes

The checksum codes we have seen (those with bit-column parity) could detect some burst errors

We can extend to an Error Correcting Code (ECC) by calculating both character and bit-column parity

M	0	100	1101
e	0	110	0101
s	1	111	0011
s	1	111	0011
a	1	110	0001
g	1	110	0111
e	0	110	0101
K	0	100	1011

1. ←
character
parity

2. ↓
bit-
column
parity

ECCs - Example

At transmitter we computer and transmit

Compute character parity bits, then bit-column parity value, then transmit message with these added bits

At receiver we re-compute, compare and correct if needed

Compute character parity bits, ignoring received parity bits

For each character, compare re-computed character parity with received parity

If not equal, ERROR somewhere in row

Re-compute bit-column parity value

Compare re-computed value with received value

If not equal, ERROR somewhere in column

If a single error detected, correct by inverting bit in message at (row, column)

ECCs - Graphically

A graphical representation allows us to think clearly about the potential issues and capabilities of a particular ECC

✓	0	0	100	1101	M e s s a g e
✓	0	0	110	0101	
✓	1	1	111	0011	
X	0	1	11 <u>0</u>	0011	
✓	1	1	110	0001	
✓	1	1	110	0111	
✓	0	0	110	0101	
		0	100	1011	
			0	101	1011
			✓✓	X	✓✓✓✓

Key:

1101

0

1011

1011

✓✓X

– received message bits

– bit changed by noise

– received parity bits

– re-computed parity bits

– comparisons

This ECC can detect and correct single errors

It can also detect, but not correct, certain combinations of multiple error

Common Memory Components

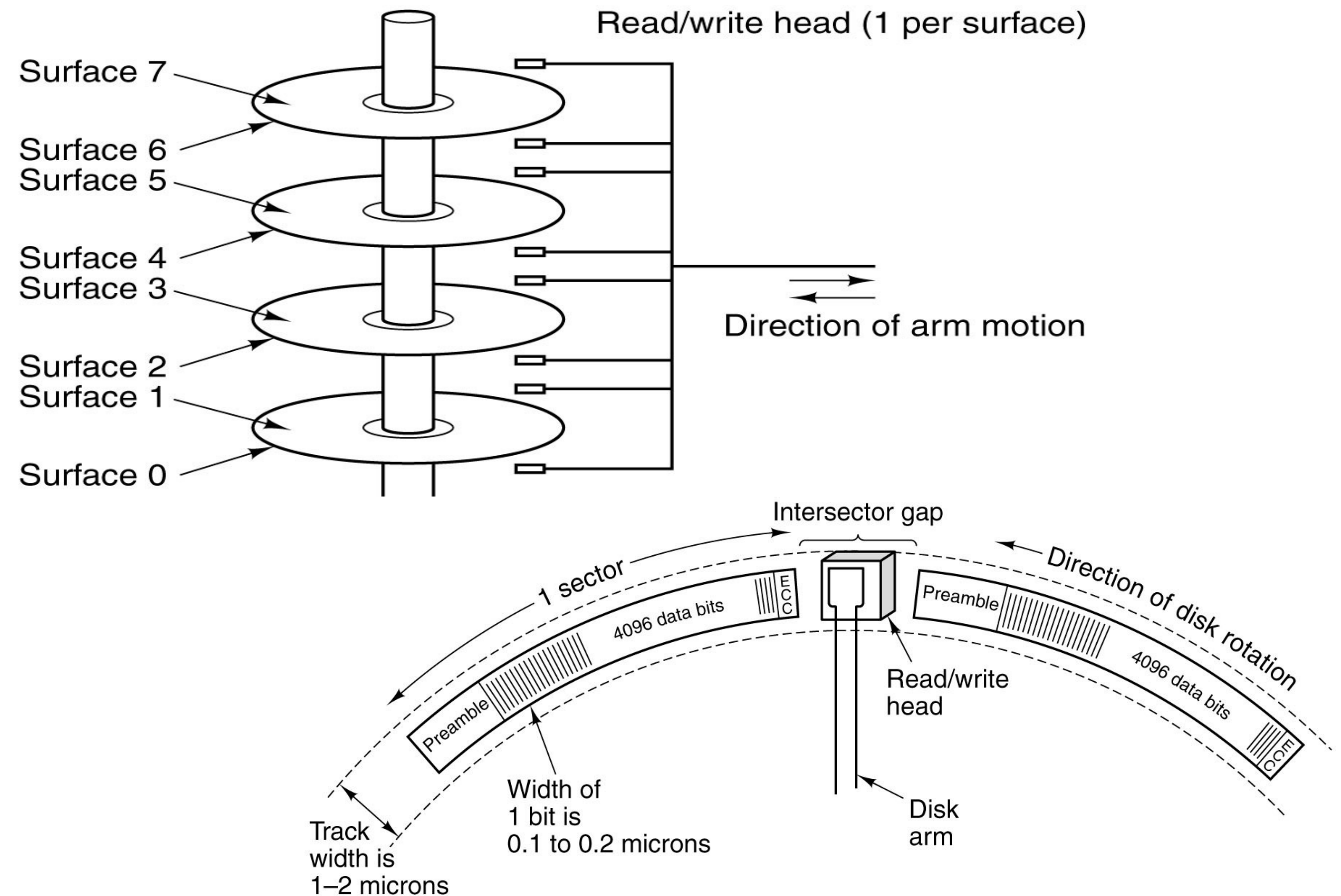
Magnetic Hard Disks - Platters, Surfaces and Heads

Record data by magnetising a thin film of ferromagnetic material on a disk

One track contains many sectors

Each sector is separated by an inter-sector gap.

Sector contains preamble to allow head to be synchronised before read/write, data and ECC



Magnetic Hard Disks - Performance - What Can We Learn?

Seek times - time to move arm to correct track

Typically about 5ms between different tracks but can be below 1ms for consecutive tracks

Rotational latency

Constant angular velocity, i.e., motor spins at constant speed

Drives available with rotational speeds of 5400 RPM, 7200 RPM or 10800 RPM

Average delay (half a rotation) = 3 to 6ms

Sector read time is about 0.013ms, which means that seek time and rotational latency dominate access time.

Formatting (preambles, ECC etc...) reduces capacity by about 15%

Hard Disks - Increasing Capacity with Zones - More To Learn?

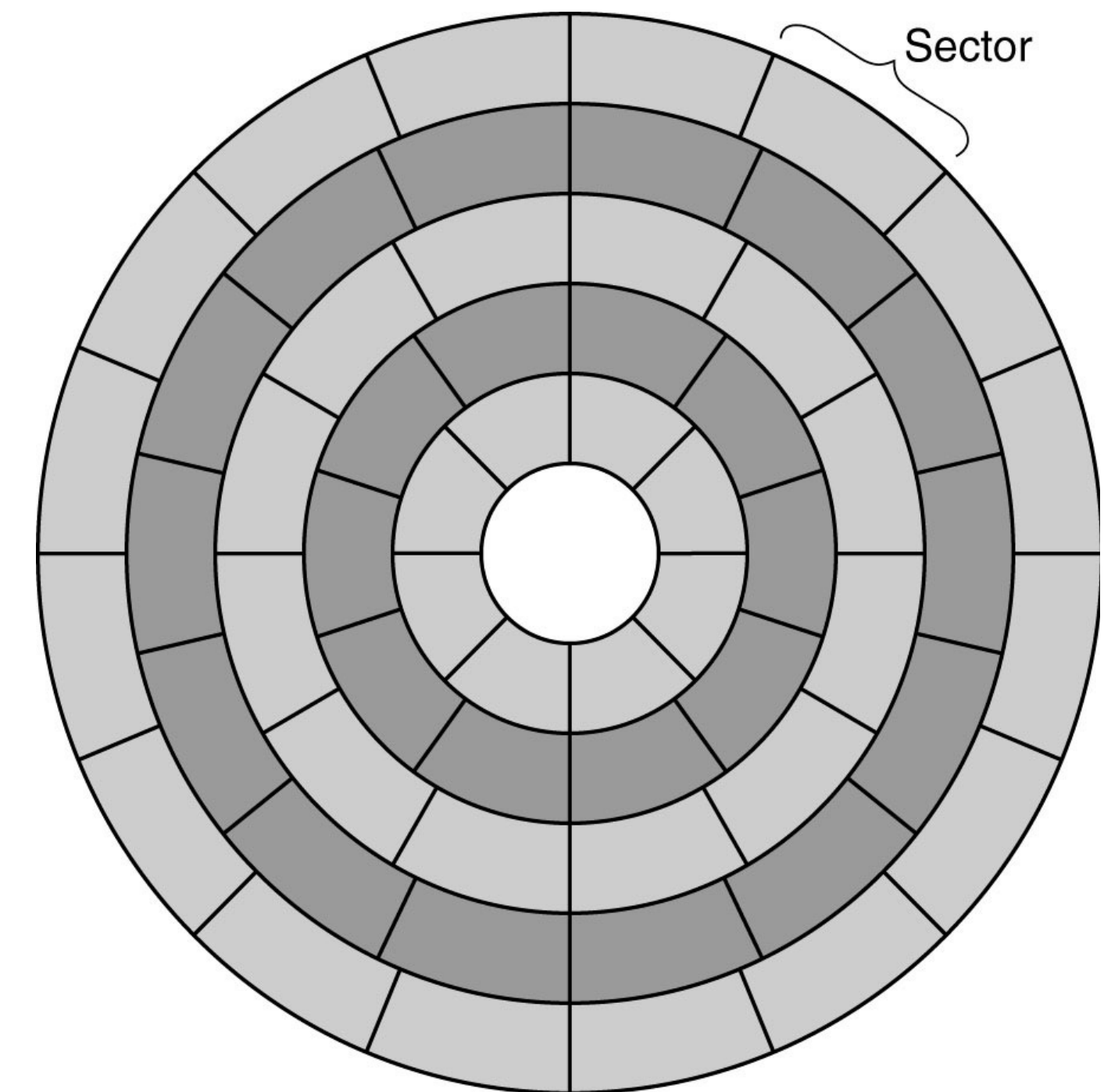
Older drives have a constant number of sectors per track, which meant that, due to constant angular velocity, bit density is lower on outer tracks

Most Recent disks are zoned (10-30 zones), with more sectors in the longer (outer) tracks

Many tracks per zone

More sectors gives higher capacity

Graphic shows a disk with 5 zones



RAID

Disks are slow, and disks are also unreliable: MTBF is now in years, but they're susceptible to power fluctuations, vibration, shock, liquids, the phase of the moon.

RAID = Redundant Arrays of Inexpensive Disk

Raid 0: Striping. Put successive blocks on successive disks, so that your reads and writes are spread evenly over the whole set. Faster, but no more reliable.

Raid 1: Mirroring. Put the entire data set on each disk, so that reads can come from either and you can survive the loss of $n-1$ disks, and you can rebuild onto a spare. Faster and more reliable.

Raid 0+1 (aka "Raid 10"): A striped volume, mirrored (or sometimes a set of mirrored volumes, striped). Or a set of mirrors, striped. Faster and more reliable, but probably won't survive as many failures.

More RAID

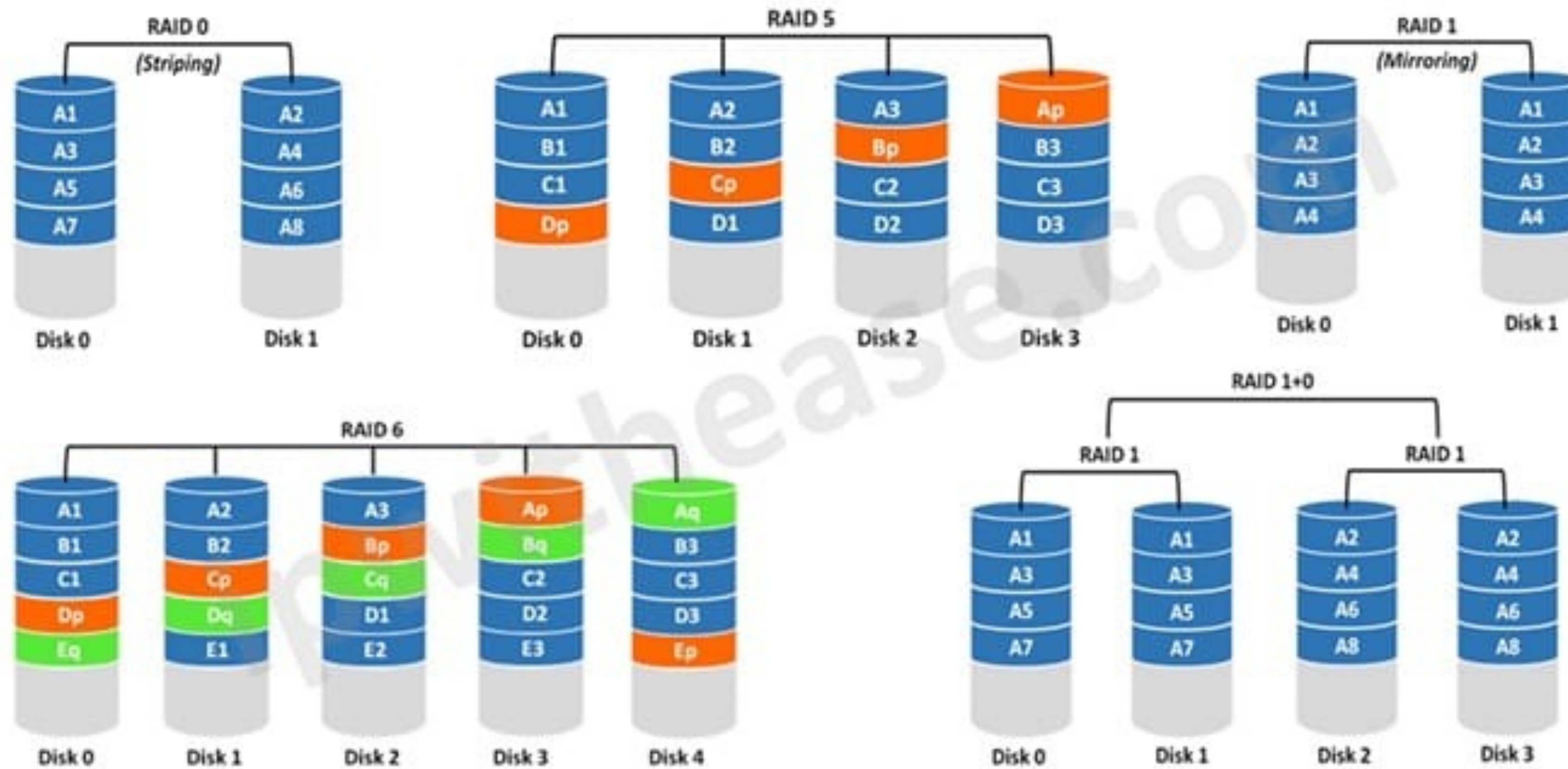
RAID 4: Stripe over $n-1$ disks, and put a parity block on the n th disk. You can then survive the loss of one disk, because you can recalculate the missing data from the parity. As fast as an $n-1$ disk stripe when all the disks are working, slow as a single spindle when in “degraded” mode, works the n th disk very hard as it has to handle write activity on every write.

RAID 5: Like RAID 4, but with the parity disk swapping on a per-cylinder or per-sector basis, so that each disk shares the load on writing. Again, slow in degraded mode, and rebuild is challenging.

RAID 6: Like RAID 5, but with two parity disks using different codes, so you can survive two disks failing.

RAID 4, 5 and 6 are popular in large disk arrays, often fronted with SSD or battery-backed RAM, because they use fewer spindles per Petabyte than RAID 0+1. Ian’s view is that unless you are building huge silos, Raid 0+1 is the way to go.

RAID Levels: RAID 0, RAID 1, RAID 5, RAID 6, RAID 10



<https://ipwithease.com>

The game changer: Solid State Disks

A solid state disk looks to the computer like a rotating magnetic disk, but is much faster.

Best case: microsecond random access ($\sim 10^6$ operations per second)

Worst case: just the same, pretty much. They usually need a pipeline of requests (ie, you can't just send a request, wait for the response, and only then send the next) but it's nothing like as critical as it is for rotating disks.

This means we don't need to worry anything like as much about sequencing, queuing, filesystem layout: it's basically an array of blocks, each the same the next. Most workloads won't get even close to the specified performance of a mechanical disk, but with SSD, it all just works. Exotic file/volume management is much less necessary

Initially, a drop-in replacement using **SATA**, savagely limited by the interface (6Gbps, <1GB/sec). Subsequently, **nVME**, which is closer, faster interface and is now hitting 15GB/sec.

So why not rip out all your rotating rust tomorrow?



Still Room for Rust

SSDs are expensive. Much, much cheaper than they were, and the ratio is not as bad, but retail of the order of 2.5x per TB.

SSDs have a write lifetime. It's not as bad as feared and there is a lot of FUD, but for some workloads it's a serious issue.

SSDs don't like being powered off. After three to six months, the data starts to disappear. For backup, MAID ("massive array of idle/inactive disks") is a thing, as is simply pulling drives out of racks.

For big, deep archival silos, **tape** is still a thing, and MAID is a slighter faster and easier to manage alternative (LTO-10 is now at 30TB/cartridge, and tape robots are fun devices)



Real storage arrays

At the front, battery-backed RAM to accelerate writes and caches frequently used reads (read-side caching less valuable than it was, because client devices now have a lot more RAM)

Behind that, flash (NVME, usually, but occasionally SATA).

Behind that, maybe fast rotating rust (15000rpm, but wear hearing protection) but less so as SSD comes down in price

Behind that, lots of slow rotating rust (7200rpm, often spun down)

Behind that, tape.

Space is an issue, but power consumption (and heat output) per square meter of floor space is often a bigger factor.



Caches on disks

An obvious optimisation is a cache.

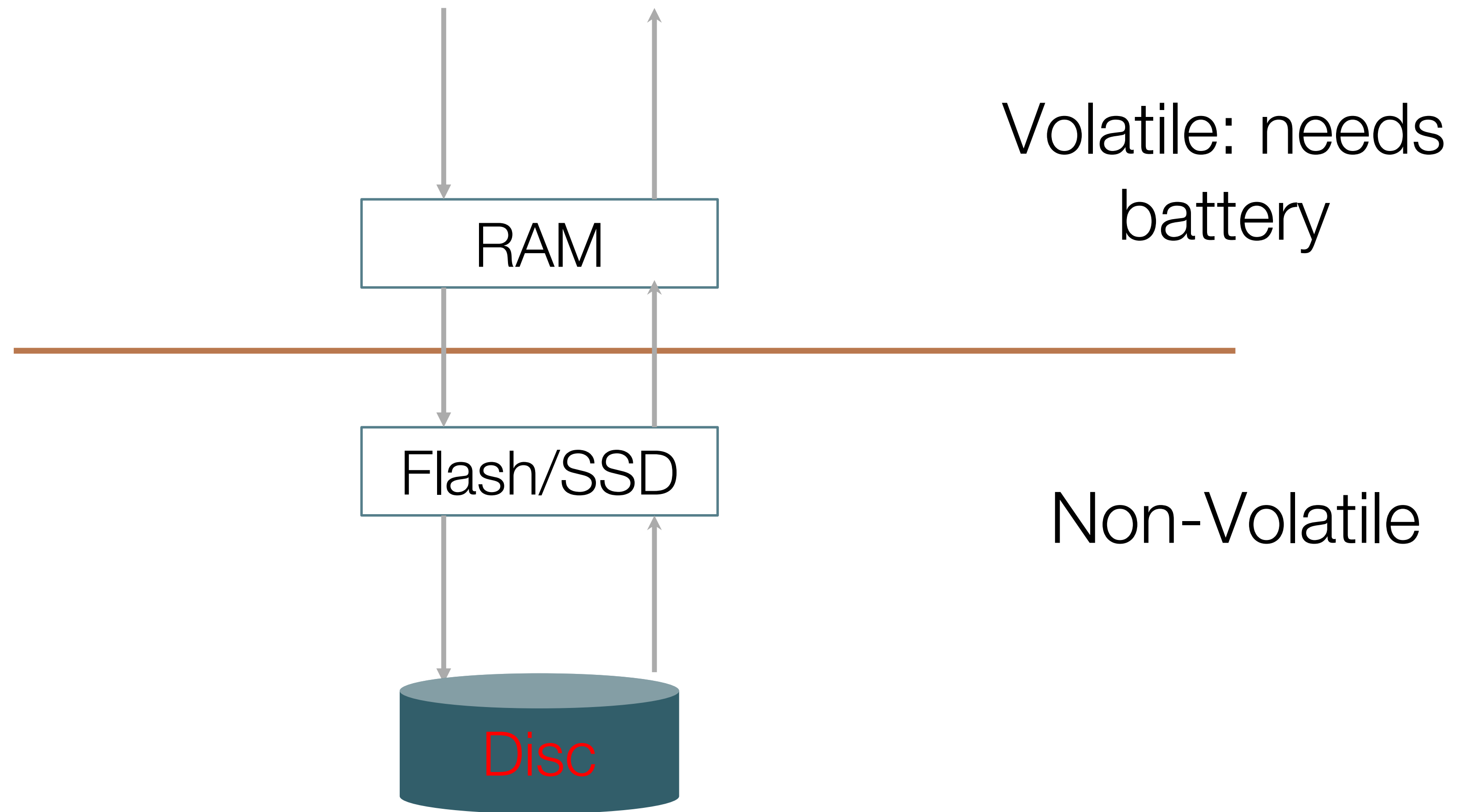
On the read side, every time you read something, read the whole track “as you are there anyway”. Keep the last few. This is transparent and harmless.

On the write side, every time you write something, write it to RAM and then write it to disk in a nice order, optimising for sequence. Many drives do this, but it is not at all clear what happens if/when the power goes off.

A compromise is “tagged queueing”, where the OS sends a sequence of writes, the drive re-orders them, but does not acknowledge them until they are safely stored away. Requires the OS be willing to write ahead (and the application actually generates such writes).

In reality, if there’s RAM and it’s not battery-backed (and stable over reset, so the whole thing can crash and restart and still flush its RAM to stable storage) it’s a danger.

The D in ACID (atomic, consistent, isolated, durable) can be hard to guarantee for production databases.



Can we do this automatically?

The answer is in layers.

Yes, the whole system can pretend it's a faster, better disk by using the cache in a default way.

Writes to go RAM and are progressively destaged to disk

Reads come from the fastest place that has them, and the caches are treated as LRU (or similar)

But, we can do better!

Filesystems can use layers

A filesystem is the software that organizes data onto a disk, providing services like directories, file modes, file names, etc. Everything that isn't "block 35628754783, please". We'll talk about this in more detail shortly.

One of the crucial jobs of filesystems is to remain consistent, even in the face of crashes and power failures.

Historic filesystems didn't, and Unix filesystems needed "fsck" (file system check) on recovery which hugely slowed recovery. Also CHKDSK, and analogues on other systems.

Journalling filesystem solved this by journalling what was about to happen, doing it, and then marking the journal as complete, thus providing an index of what needs to be checked.

Databases do similar things: record the transaction you're about to do, do it, mark it as done, allows transactions to be done atomically.

The benefits of a storage hierarchy

Use fast SSD or battery-backed RAM for the journal: things happen quickly.

Stage the actual writes through to slower SSD and rotating disk: capacity and cheap.

The long-term storage volatility doesn't matter, as the data will only remain in the journal for a short period (seconds, at most). Whether or not loss of the contents of the journal results in corruption, data loss or the need to consistency check a lot of data (which are escalatingly serious) is down to the detailed design.

But mirroring, multi-way, the journal is easy as it's small.

You may also get "hot backup", particularly for large databases, where the journal stores changes while the database is locked. Giving rise to the best backup strategy for large databases...

Database hot backup

Put the database on a three-way mirror (ie, three copies of the database on three separate devices)

When you want to back up, tell the database to put transactions into the journal and **not** flush them to the main database, so that the main database is now quiescent.

As soon as that mode is engaged, drop one of the mirrors off. You now have a separate, consistent copy you can backup to tape or other device at full speed, without impacting the main database in service.

When you finish, resynchronise the mirror (there are various techniques which optimize this, involving recording which blocks were changed, but in the worst case it will consume 50% of the bandwidth of each of the live copies to resynch at 100% of the third copy's performance).

This is the way.

How do we use the storage?

The conceptual view from a program is (in 2025) almost entirely Unix/Linux/POSIX oriented.

There have been other filesystem structures, and Dave Cutler's work at DEC (RSX11, VAX/VMS) heavily influenced Dave Cutler's work at Microsoft (everything since NT, up to Windows 11 and its server variants).

Since NT's predecessors MS-DOS and Windows 1/2/3 were influenced by RSX11 and other DEC operating systems, particularly TOPS-10, there's a whole "alternative world" of filesystems we can imagine, which include wonderful things like versioning (you write to MYFILE.TXT;1 and get a fresh MYFILE.TXT;2, all for free!)

But Unix won out, not always for good reasons: even as a Unix fanboi I recommend the essay "**Worse is Better**" by Richard P Gabriel.

The Unix View

“Everything’s a file” (except for the things that aren’t).

User files: /Users/igb/myfile.txt, /home/igb/myfile.txt

Binaries: /Applications/Emacs.app, /usr/bin/emacs

Configuration files: /etc/resolv.conf, /etc/network/interfaces

Temporary and local files: /var/cache

Devices: /dev/tty, /dev/fd/0, /dev/dsk/c0t1d0s1

Configuration and control: /sys, /proc

In an ideal world, we would also have

Net: /net/tcp/[www.google.com/https](https://www.google.com/)

And the reasons why this wasn’t actually done are the one of the things that “Worse is Better” is addressing: it was the Right Thing, but it was harder, so it didn’t get done, and by the time it was done the caravan had moved on.

The Unix View

Everything in the filesystem is manipulated with `open()`, `close()`, `read()` and `write()`, with `ioctl()` ("I/O Control") and `fcntl` ("File Control") used to do specialised things to devices, once they have been `open()`ed.

(Callback to bitwise OR: note use of bitwise OR operation `|` to combine flags into a single set of flags).

```
igb@mail-nuc:~$ cat hw.c
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h> // only for perror

int
main (int argc, char **argv) {
    char *s = "hello world\n";
    int fd = open ("file.txt", O_WRONLY | O_CREAT | O_EXCL);
    if (fd < 0) {
        perror ("open");
        exit (1);
    }
    if (write (fd, s, strlen(s)) != strlen(s)) {
        perror ("write");
        exit (1);
    }
    (void) close (fd); // don't do this in production code
}
igb@mail-nuc:~$ cc -o hw hw.c
igb@mail-nuc:~$ ./hw
igb@mail-nuc:~$ cat file.txt
hello world
igb@mail-nuc:~$ ./hw
open: File exists
igb@mail-nuc:~$
```


What does this look like on disk?

Underneath the filesystem, we have an array of blocks.

Originally, each filesystem lived in a disk partition: a region of disk defined when the disk was formatted, containing a set of blocks numbered zero upwards. This meant the largest filesystem you could have was the size of one disk drive, and conventionally Unix/Linux machines had a set of partitions for /, /usr, /etc, /var and so on.

Filesystems are “mounted” by taking a partition or something which pretends to be a partition and mapping it into a part of the filesystem. Linux/MacOS you can see this with the “mount” command, no arguments, although most of what that will show you today are various pseudo filesystems.

```
igb@mail-nuc:~$ mount | grep ext4
/dev/md0 on / type ext4 (rw,relatime,errors=remount-ro)
/dev/sdc1 on /backups type ext4 (rw,relatime,errors=remount-ro)
igb@mail-nuc:~$
```

What does this look like on disk

In 2025, a lot of constraints have been removed.

- * We are not limited to 2GB per filesystem, because everything's 64 rather than 32 bits
- * We are not limited by how quickly we can recover the data after a crash, because journalling
- * We are not limited by the size of the filesystem in other ways, coming up, because hash tables and other good data structures
- * We like RAID, because the reliability is nice (even with SSD)
- * So...

Volume Management

One solution (Linux “md” subsystem) is to have a facility to group physical devices into the various RAID groupings, to produce an arbitrarily large “volume”, and then pretend that is a disk. Here, a simple RAID 1 mirror over two devices.

```
igb@mail-nuc:~$ sudo /usr/sbin/mdadm --detail /dev/md0
/dev/md0:
```

```
Version : 1.2
Creation Time : Tue Oct  1 21:27:03 2019
Raid Level : raid1
Array Size : 233530368 (222.71 GiB 239.14 GB)
Used Dev Size : 233530368 (222.71 GiB 239.14 GB)
Raid Devices : 2
Total Devices : 2
Persistence : Superblock is persistent

Intent Bitmap : Internal

Update Time : Sun Nov 23 10:58:40 2025
State : clean
Active Devices : 2
Working Devices : 2
Failed Devices : 0
Spare Devices : 0
```

Consistency Policy : bitmap

```
Name : mail-nuc:0
UUID : 808e15fe:39ee02c5:d9663408:8af6c181
Events : 14367
```

Number	Major	Minor	RaidDevice	State		
0	8	3	0	active	sync	/dev/sda3
1	8	19	1	active	sync	/dev/sdb3

Filesystems on arrays of blocks

However we get our array of blocks, we then want to put a filesystem on it.

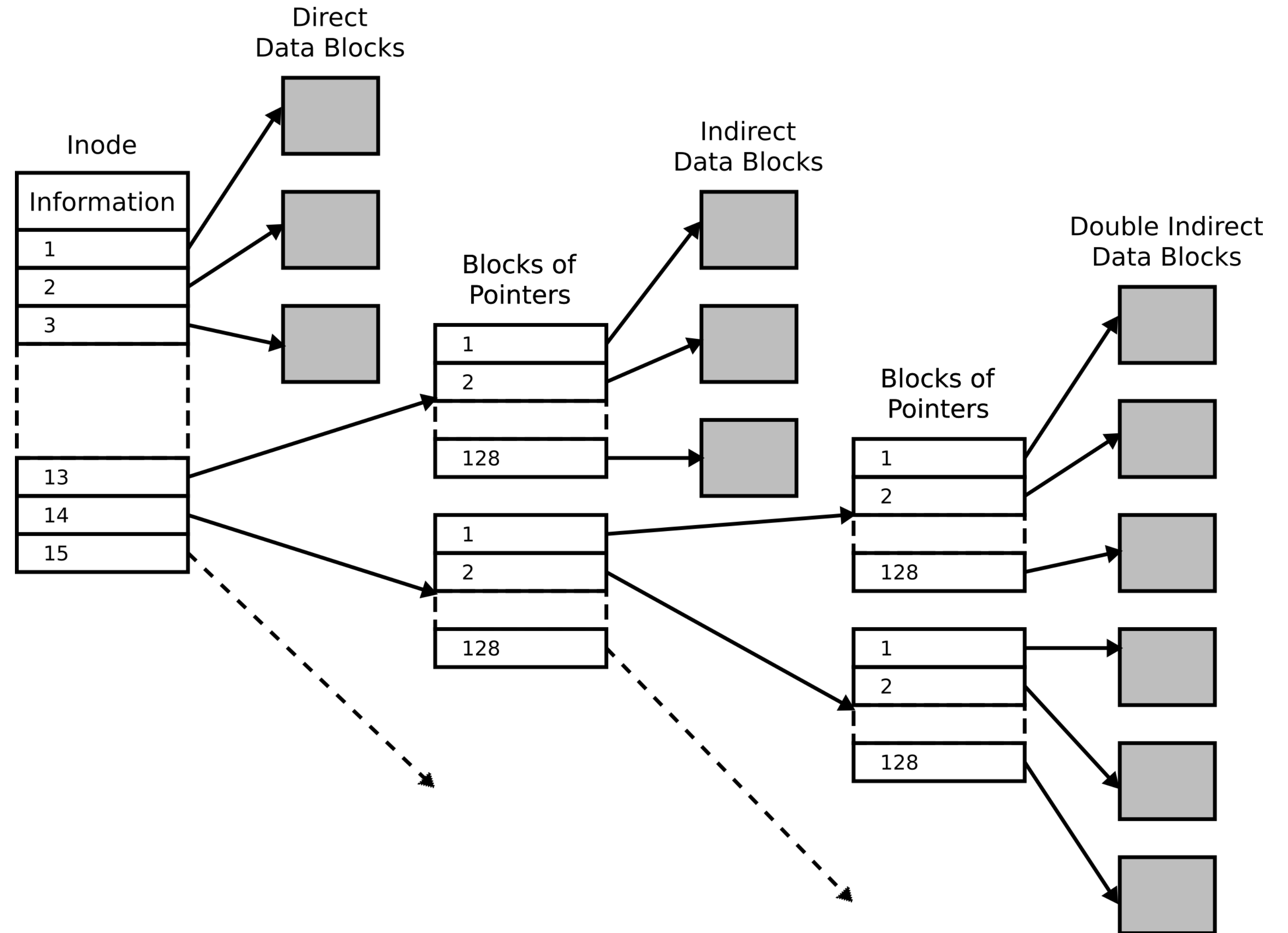
Filesystems are complex, scary beasts. There have only been a handful which have achieved broad acceptance (original Unix ufs, Berkeley Fast File System, ext2/3/4, arguably btrfs) but most of the rest are proprietary and closely tied to their platforms (HFS+, APFS, XFS, VxFS), have complex licensing issues which scare people (the fantastic ZFS) or are controversial in other ways (bcachefs has recently been ejected from the Linux kernel due to the behaviour of the developers, for example, btrfs has persistent reliability problems).

A full-fat modern filesystem is far too complex to unpick in a brief set of slides. I'm going to talk about an idealised version of the original Unix filesystem, and point to where the problems are.

Traditional Unix Filesystem

There are two sorts of blocks on disk: data blocks, and inodes ("index nodes"). They're set up as one or the other on initialisation.

The index nodes contain some information at the top (a type, and information specific to that type of nodes) and, where relevant, pointers to data blocks. The further into the file, the longer the path to the block.



(Wikipedia)

What's in the inode?

The pointers to the data.

The type (file, directory, device, a menagerie of other things)

If it's a device, information about which device it is ("major device number" is its type, "minor device number" is an instance of that device)

Permissions

Modification Times

Sizes, both in terms of blocks and bytes (attempts are made to not allocate all-zero blocks)

What's not in the inode?

The name.

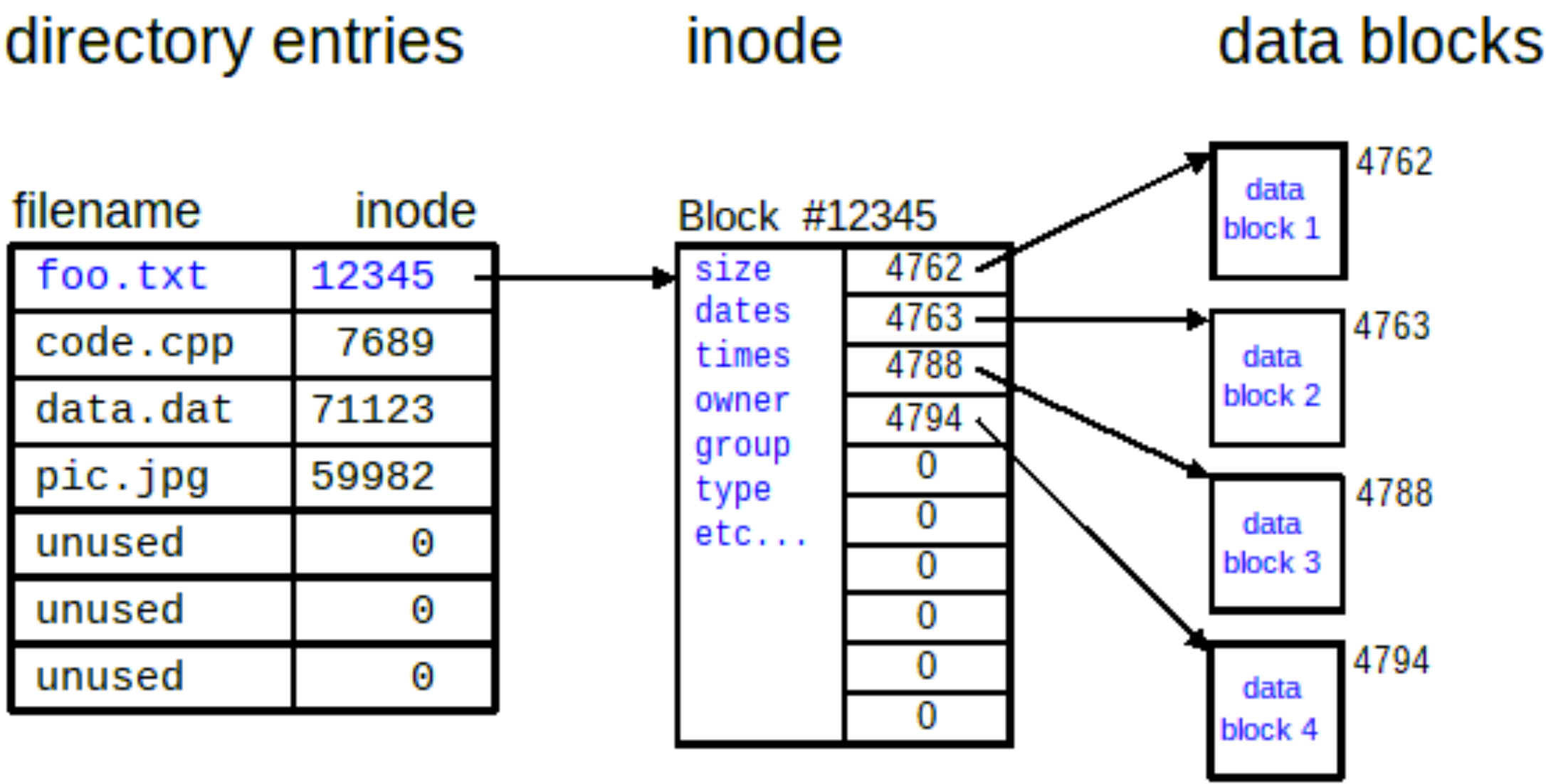
Files (directories, devices) don't have names, in a Unix filesystem. They exist anonymously as inodes.

A directory contains a set of name/inode mappings, itself consisting of an inode, and a datablock containing the mappings.

When you open() a file, you are getting a reference to the in-memory version of the inode.

Multiple names in multiple directories can point to the same file, of which there is only one copy.

Why do you think we delete files with the unlink() system call?



<https://azrael.digipen.edu/~mmead/www/Courses/CS180/FileSystems-1.html>

Directories

Original version is an array of (name, inode) pairs, and explains why older filesystems have limits on the length of filenames (old Unix was 14 byte filenames with two bytes for an inode).

Problem one: we'd like arbitrary length filenames. Fixed in Berkeley FFS.

Problem two: we'd like `ls -l` and similar commands to be linear time, rather than requiring the directory to be scanned to find the file for each filename.

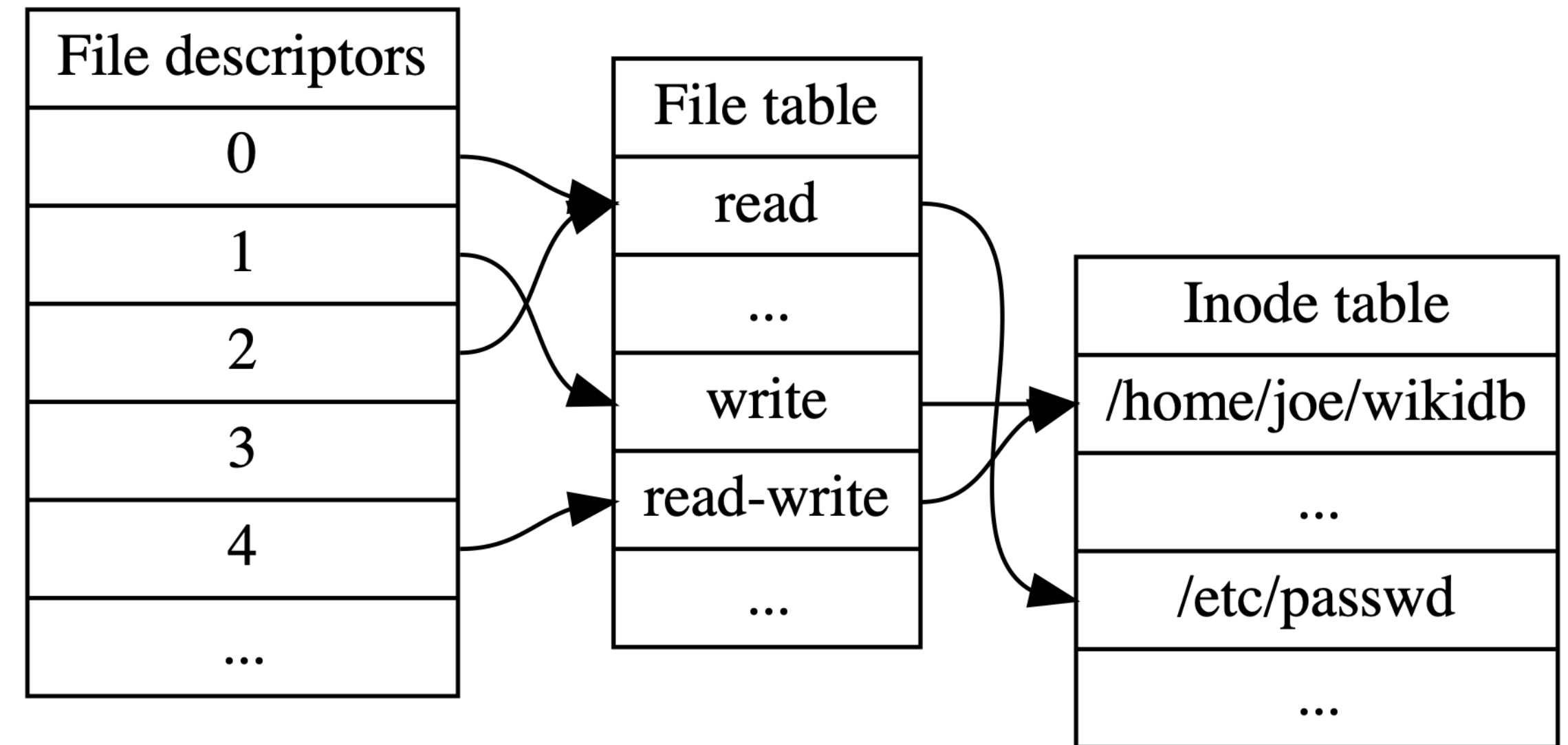
Tactical solution: cache the last filename returned and start the search at that point, so a sequential scan of the files in a directory is fast

Strategic solution: hash table ("dictionary") for directory names.

So why can we delete files and continue using them?

inodes contain a reference count, both on disk ("how many directories point to me?") and in memory ("how many file descriptors point to me?")

Files are only reclaimed (inode marked unused, data blocks put on free list) when both reference counts are zero.



If you create a file, open it, and then immediately unlink it, the on-disk reference count is zero, but the in-kernel reference count is 1.

If your program dies unexpectedly, both hit zero, and the file is cleaned up. Much easier than relying on exception handlers.

Inconsistency and Journalling

Unlinking a file has two steps: mark the directory entry as unused (so ls doesn't show it) and decrement the reference count in the inode (so the space is reclaimed when it hits zero).

If you decrement the counter on a file that is still present in a directory, then it looks like it's there until you open it, when it fails. If you remove it from the directory without decrementing the counter, the space is potentially left allocated forever. Both can happen if there is a crash during the critical region.

Old solution: hope it works, and then analyse the whole filesystem for such inconsistent situations after an unclean shutdown, plus once in a while on general principles ("fsck")

New solution: record you're about to do a thing, do a thing, record that you've done a thing. Use a separate, fast piece of storage so it happens in parallel. "Replaying the journal" then consists of just checking the things that aren't marked as complete.

Read ahead, write behind, caching

Read ahead: predict what they'll want next (often, "the rest of the file") do it, and cache the results.

Write behind: don't perform writes at the point of writing, but instead cache them in the hope that we can perform a smoother, more continuous write, or perhaps the write is itself overwritten.

Unix: "sync" is called every 30 (or 10, or 60, or something) seconds, "fsync" will synchronise a particular file, (usually) files are implicitly sync'd on close(),

you can also open files in mode O_SYNC and have the writes go to disk when performed (often buggy, always slow) and a variety of other kernel modes for special cases, but only do this if you know what you are doing, and anyone clever enough to do it is largely clever enough not to do it.

This gets massive performance benefits on rotating disks, considerable benefits even on SSD. But...

What goes wrong?

Disks lie. All the time. They lie and they lie and they lie.

Most importantly, they lie about “did you actually perform this write?” which makes the sync routines somewhat tricky.

In particular, 2.5” laptop drives very often assume the power won’t go off, or at least they’ll get some warning, and lie about all synchronous writes. Instead, they just put the data in RAM and pinky-promise that they’ll write it later. A lot of enterprise kit in 2025 uses 2.5” drives, because they’re lower power, fast enough, and denser.

[[Some manufacturers of file servers go to the lengths of putting custom firmware on disk drives to avoid this.]]